SISTEMAS DIGITAIS II - 2026/27

TECNOLOGIA SETÚBAL

ESCOLA SUPERIOR POLITÉCNICO SETÚBAL

Laboratório Nº	1
Data	09/10/2026
Docente	José Sousa
Curso	LEEC
Turma	PL1

Alunos		
	202300553	Rodrigo Silva Costa
	2025184378	Diego Alejandro Castro Laya

1.º Trabalho de Laboratório

Gestão do display da placa Basys3

(Tutorial Digital e Vivado)

Índice

1 Introdução....2
2 Multiplexer 4x1....4
2.1 Desenho do circuito no Digital....4
2.2 Simulação no Digital....4
2.3 Exportação para Verilog....5
2.4 Projeto no Vivado e ficheiro de restrições....6
2.5 Testbench e simulação....6
2.6 Geração do bitstream e ensaio na Basys3....9
3 Descodificador 2x4 com enables individuais....10
4 Multiplexer 16x4....15
5 Transcodificador BCD para display de 7 segmentos....20
5.1 Ensaio do transcodificador na placa....23
6 Registo de 4 bits com clear e enable....26
7 Divisor de clock e contador de módulo 4....29
7.1 Divisor de clock....29
7.2 Contador de módulo 4....31
8 Controlador dos displays da placa Basys3....34
8.1 Circuito de teste na placa....40
9 Análise crítica do código de fontes externas....48
9.1 Código gerado pelo Digital....48
9.2 Transcodificador do Moodle....48
9.3 Divisores de clock do Moodle....48
9.4 Contadores do Moodle....49
9.5 Primitiva FDCE....49
9.6 IA generativa....49
10 Conclusão....51
Anexo A: Organização dos ficheiros....52

1 Introdução

No âmbito da unidade curricular de Sistemas Digitais II, o presente relatório descreve o desenvolvimento do 1.º Trabalho de Laboratório, cujo objetivo é projetar o controlo do *display* de 7 segmentos e 4 dígitos da placa Basys3 e, com isso, percorrer o fluxo completo de projeto na ferramenta Vivado da Xilinx (licença WebPACK). Utilizou-se a versão 2026.1 do Vivado e a versão v0.31 do Digital, de Helmut Neemann.

O trabalho foi realizado em três aulas, seguindo a ordem do enunciado:

- **Aula 1**: desenho, simulação e implementação de um *multiplexer* 4x1, de um descodificador 2x4 com *enables* individuais e de um *multiplexer* 16x4 construído por hierarquia de módulos a partir do *multiplexer* 4x1;
- **Aula 2**: desenho e simulação de um registo de 4 *bits* com *clear* e *enable* (baseado na primitiva FDCE), de um transcodificador para *display* de 7 segmentos, de um divisor de *clock* e de um contador de módulo 4;
- **Aula 3**: integração de todos os módulos no controlador dos quatro *displays* (Fig. 31 do enunciado) e no circuito de teste para a placa (Fig. 32 do enunciado).

Os módulos mais simples foram desenhados na aplicação Digital, simulados, exportados para Verilog e importados para o Vivado, onde foram novamente simulados (simulação comportamental), sintetizados, implementados e convertidos em *bitstream*. Os restantes módulos foram escritos diretamente em Verilog no Vivado. Para cada módulo apresentam-se o código, o *testbench*, o ficheiro de restrições (quando aplicável) e os resultados da simulação, devidamente comentados.

Todos os *testbenches* verificam automaticamente as saídas da unidade em teste (UUT) e terminam com a mensagem `Simulacao OK` ou `Simulacao FALHOU`, pelo que o resultado não depende apenas da inspeção visual das formas de onda. A organização dos ficheiros entregues está descrita no Anexo A.

2 Multiplexer 4x1

2.1 Desenho do circuito no Digital

O *multiplexer* 4x1 foi desenhado na aplicação Digital com quatro portas AND de 3 entradas e uma porta OR de 4 entradas (Figura 1). Cada porta AND corresponde a uma das combinações das variáveis de seleção: cada variável de seleção entra negada nas portas AND em que vale 0, de modo que, para cada combinação, apenas uma das portas AND pode deixar passar a respetiva entrada de dados (DataIn0 a DataIn3). A saída DataOut é a soma lógica das quatro portas:

$$DataOut = Sel1' \cdot Sel0' \cdot DataIn0 + Sel1' \cdot Sel0 \cdot DataIn1 + Sel1 \cdot Sel0' \cdot DataIn2 + Sel1 \cdot Sel0 \cdot DataIn3$$

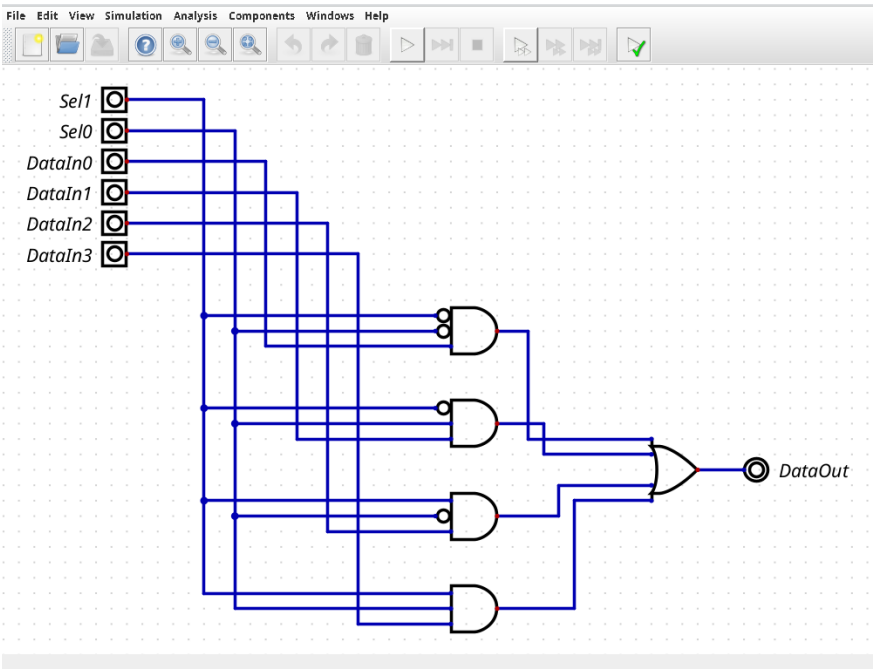


Figura 1: Circuito final do multiplexer 4x1 desenhado no Digital (`digital/multiplexer4x1.dig`).

2.2 Simulação no Digital

O circuito foi testado no simulador do Digital, comutando as entradas e observando a saída. A Figura 2 mostra um dos casos testados: com Sel1 = 1 e Sel0 = 0 está selecionada a entrada DataIn2; como DataIn2 = 1, a saída fica a 1 (a verde-claro), apesar de DataIn0 também estar a 1, o que mostra que as entradas não selecionadas não chegam à saída.

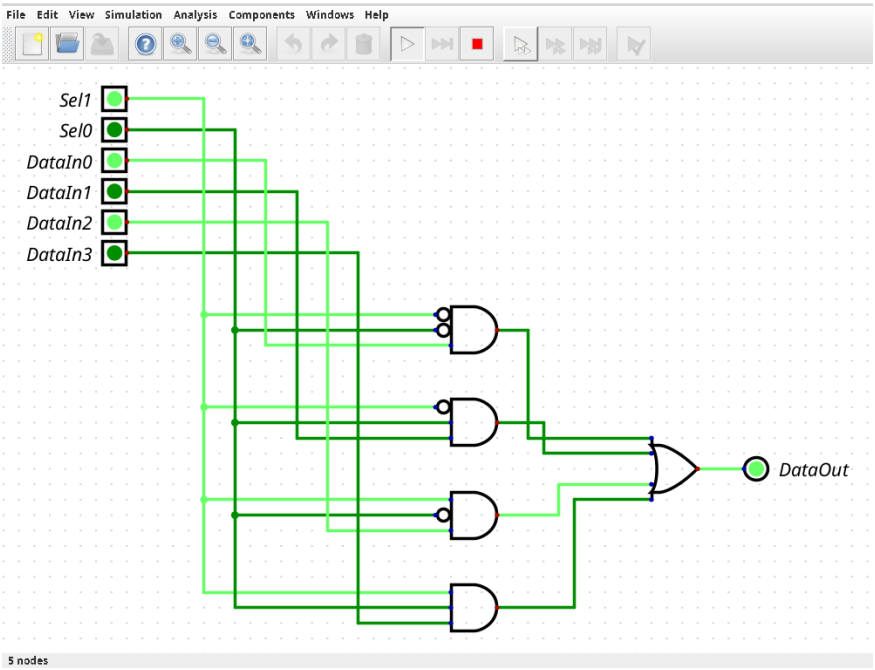


Figura 2: Simulação do multiplexer 4x1 no Digital (Sel1 = 1, Sel0 = 0, DataIn0 = DataIn2 = 1, DataOut = 1). As linhas a verde-claro estão a 1 e as linhas a verde-escuro estão a 0.

Repetiu-se o ensaio para as quatro combinações das variáveis de seleção, colocando a 1 ora a entrada selecionada, ora as restantes. A Tabela 1 resume, a partir destes testes, qual das entradas é representada na saída para cada combinação das variáveis de seleção.

Tabela 1: Entrada representada na saída do multiplexer 4x1 em função da seleção.

Sel1	Sel0	Entrada selecionada	DataOut
0	0	DataIn0	DataIn0
0	1	DataIn1	DataIn1
1	0	DataIn2	DataIn2
1	1	DataIn3	DataIn3

2.3 Exportação para Verilog

O circuito foi exportado através de File > Export > Export to Verilog, o que gerou o ficheiro `multiplexer4x1.v` (Listagem 1). O código gerado pelo Digital é uma descrição **comportamental**: uma única atribuição contínua (`assign`) descreve a função do circuito, sem instanciar portas lógicas nem definir as ligações entre elas. Os nomes das portas do módulo são os nomes atribuídos às entradas e saídas no esquema; por isso foi importante usar nomes simples, sem espaços nem acentos, e ter atenção às maiúsculas, já que o Verilog distingue maiúsculas de minúsculas.

Listagem 1: Código Verilog do multiplexer 4x1 gerado pelo Digital (`multiplexer4x1/multiplexer4x1.v`)

```
1  /*
2   * Generated by Digital. Don't modify this file!
3   * Any changes will be lost if this file is regenerated.
4   */
5
6  module multiplexer4x1 (
7    input Sel1,
8    input Sel0,
9    input DataIn0,
10   input DataIn1,
11   input DataIn2,
12   input DataIn3,
```

```
  output DataOut
);
  assign DataOut = ((~ Sel1 & ~ Sel0 & DataIn0) | (~ Sel1 & Sel0 & DataIn1) | (Sel1 & ~ Sel0 & DataIn2) | (Sel1 & Sel0 & DataIn3));
endmodule
```

2.4 Projeto no Vivado e ficheiro de restrições

No Vivado foi criado um projeto RTL (File > Project > New) com o nome `multiplexer4x1`, para a FPGA da Basys3 (`xc7a35ticpg236-1L`), ao qual se adicionou o ficheiro exportado. O ficheiro de restrições `multiplexer4x1.xdc` (Listagem 2) parte do indicado no enunciado, que associa as variáveis de seleção a dois interruptores, as entradas de dados a quatro teclas e a saída a um LED. No laboratório optou-se por ligar também as entradas de dados a interruptores (SW0 a SW3), com a seleção em SW5 e SW4 e a saída no LED LD0 (Tabela 2). Assim, o estado de todas as entradas fica visível ao mesmo tempo, sem ser preciso manter teclas premidas. As duas últimas linhas definem a tensão de configuração da placa (3,3 V).

Tabela 2: Ligação dos sinais do multiplexer 4x1 aos pinos da FPGA.

Sinal	Pino da FPGA	Elemento da Basys3
Sel1 / Sel0	V15 / W15	Interruptores SW5 / SW4
DataIn0 / DataIn1	V17 / V16	Interruptores SW0 / SW1
DataIn2 / DataIn3	W16 / W17	Interruptores SW2 / SW3
DataOut	U16	LED LD0

Listagem 2: Ficheiro de restrições do multiplexer 4x1 (`multiplexer4x1/multiplexer4x1.xdc`)

```
# Variáveis de seleção do multiplexer 4x1 (interruptores SW5 e SW4)
set_property PACKAGE_PIN V15 [get_ports Sel1]
    set_property IOSTANDARD LVCMOS33 [get_ports Sel1]
set_property PACKAGE_PIN W15 [get_ports Sel0]
    set_property IOSTANDARD LVCMOS33 [get_ports Sel0]

# Variáveis de entrada do multiplexer 4x1 (interruptores SW0 a SW3)
set_property PACKAGE_PIN V17 [get_ports DataIn0]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn0]
set_property PACKAGE_PIN V16 [get_ports DataIn1]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn1]
set_property PACKAGE_PIN W16 [get_ports DataIn2]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn2]
set_property PACKAGE_PIN W17 [get_ports DataIn3]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn3]

# Variável de saída do multiplexer 4x1 (LED LD0)
set_property PACKAGE_PIN U16 [get_ports DataOut]
    set_property IOSTANDARD LVCMOS33 [get_ports DataOut]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

2.5 Testbench e simulação

O *testbench* `multiplexer4x1_teste1.v` (Listagem 3) segue a estrutura do enunciado: declara a seleção `S` e os dados `D` como `reg`, a saída `Z` como `wire`, instancia o *multiplexer* com o nome UUT e altera as entradas num ciclo `for` com um `case`, de 50 em 50 ns, depois de esperar 200 ns pela inicialização da FPGA.

A sequência foi personalizada para ilustrar melhor o funcionamento do circuito. Para cada valor da seleção, coloca-se primeiro a 1 **apenas** a entrada selecionada (Z deve ser 1) e depois colocam-se a 1 **todas as outras** entradas, mantendo a selecionada a 0 (Z deve ser 0). Assim, fica demonstrado que a saída acompanha a entrada selecionada e que nenhuma das restantes tem influência. No fim, com todas as entradas a 1, a seleção percorre os valores 2, 1 e 0 (Z mantém-se a 1). Os valores esperados estão na Tabela 3.

Além disso, um bloco `always` compara automaticamente, 1 ns depois de cada mudança das entradas, a saída `Z` com `D[S]` e conta os erros; no fim é escrita na consola do simulador a mensagem com o resultado.

Tabela 3: Sequência aplicada no testbench do multiplexer 4x1 e valores esperados de Z.

Instante (ns)	S	D (DataIn3..0)	Z esperado	Observação
0	00	0000	0	Estado inicial
250	00	0001	1	Só a entrada 0 a 1
300	00	1110	0	Todas a 1 menos a 0
350	01	1110	1	Seleciona a entrada 1
400	01	0010	1	Só a entrada 1 a 1
450	01	1101	0	Todas a 1 menos a 1
500	10	1101	1	Seleciona a entrada 2
550	10	0100	1	Só a entrada 2 a 1
600	10	1011	0	Todas a 1 menos a 2
650	11	1011	1	Seleciona a entrada 3
700	11	1000	1	Só a entrada 3 a 1
750	11	0111	0	Todas a 1 menos a 3
800	11	1111	1	Todas a 1
850 / 900 / 950	10 / 01 / 00	1111	1	Seleção percorre 2, 1 e 0
1000	00	0000	0	Todas a 0

Listagem 3: Testbench do multiplexer 4x1 (`multiplexer4x1/multiplexer4x1_teste1.v`)

```
1  `timescale 1ns / 1ps
2
3  // Testbench do multiplexer 4x1 (sequência personalizada)
4  module multiplexer4x1_teste1();
5
6  //   Declaração das entradas
7       reg [1:0] S;
8       reg [3:0] D;
9
10 //   Declaração da saída
11      wire Z;
12
13 //   Instanciação da UUT e ligação das entradas e saídas
14      multiplexer4x1 UUT (
15          .Sel1(S[1]),
16          .Sel0(S[0]),
17          .DataIn0(D[0]),
18          .DataIn1(D[1]),
19          .DataIn2(D[2]),
20          .DataIn3(D[3]),
21          .DataOut(Z)
22      );
23
24 //   Declaração de vars auxiliares
25      integer n;            // Contador para o ciclo for
```

```
26      integer erros = 0;    // Número de saídas diferentes do esperado
27
28  //  Verificação automática: a saída deve ser sempre igual à entrada selecionada
29      always @(S or D) begin
30          #1;
31          if (Z !== D[S]) begin
32              $display("ERRO t=%0t: S=%b D=%b Z=%b (esperado %b)", $time, S, D, Z, D[S]);
33              erros = erros + 1;
34          end
35      end
36
37  //  Definição do valor das entradas ao longo do tempo
38  //  Para cada seleção: liga só a entrada selecionada (Z=1), depois liga todas
39  //  as outras exceto a selecionada (Z=0), mostrando que só a selecionada passa.
40      initial begin
41          S = 2'b00;        // Valor inicial das vars de seleção
42          D = 4'b0000;      // Valor inicial das vars de entrada (Z=0)
43
44          #200;             // Aguardar 200ns para simular o tempo
45                            // das configurações iniciais da FPGA
46
47          for (n=0; n<16; n=n+1)
48          begin
49              #50;                          // Passam 50ns a cada ciclo de n
50              case (n)
51                   0: D = 4'b0001;          // Entrada 0 a 1, selecionada a 0 (Z=1)
52                   1: D = 4'b1110;          // Todas a 1 menos a 0 (Z=0)
53                   2: S = 2'b01;            // Seleciona a entrada 1 (Z=1)
54                   3: D = 4'b0010;          // Só a entrada 1 a 1 (Z=1)
55                   4: D = 4'b1101;          // Todas a 1 menos a 1 (Z=0)
56                   5: S = 2'b10;            // Seleciona a entrada 2 (Z=1)
57                   6: D = 4'b0100;          // Só a entrada 2 a 1 (Z=1)
58                   7: D = 4'b1011;          // Todas a 1 menos a 2 (Z=0)
59                   8: S = 2'b11;            // Seleciona a entrada 3 (Z=1)
60                   9: D = 4'b1000;          // Só a entrada 3 a 1 (Z=1)
61                  10: D = 4'b0111;          // Todas a 1 menos a 3 (Z=0)
62                  11: D = 4'b1111;          // Todas a 1 (Z=1)
63                  12: S = 2'b10;            // Seleciona a entrada 2 (Z=1)
64                  13: S = 2'b01;            // Seleciona a entrada 1 (Z=1)
65                  14: S = 2'b00;            // Seleciona a entrada 0 (Z=1)
66                  15: D = 4'b0000;          // Todas a 0 (Z=0)
67              endcase
68          end
69          #50;
70          if (erros == 0) $display("Simulacao OK: multiplexer4x1 sem erros");
71          else            $display("Simulacao FALHOU: %0d erros", erros);
72          $finish;
73      end
74  endmodule
```

A Figura 3 mostra o resultado da simulação comportamental no Vivado. Os barramentos `S` e `D` foram expandidos para se ver cada *bit* individualmente. Verifica-se que a saída `Z` corresponde, em todos os intervalos, aos valores da Tabela 3: fica a 1 sempre que a entrada selecionada está a 1 e a 0 quando apenas as outras entradas estão a 1. A consola do simulador apresentou `Simulacao OK: multiplexer4x1 sem erros`.

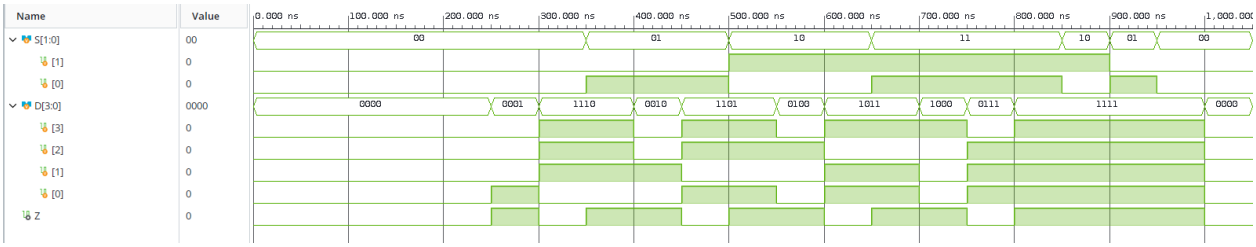


Figura 3: Simulação comportamental do multiplexer 4x1 no Vivado.

2.6 Geração do bitstream e ensaio na Basys3

Depois de confirmar o ficheiro de restrições, executaram-se, com as opções por omissão, a síntese (Run Synthesis), a implementação (Run Implementation) e a geração do *bitstream* (Generate Bitstream), sem erros. O circuito ocupa 1 LUT da FPGA, o que é coerente com uma função de 6 variáveis, que cabe numa única LUT de 6 entradas da Artix-7. O ficheiro `multiplexer4x1.bit` é carregado na placa com a aplicação Adept da Digilent (botão Program); a programação termina com a mensagem *Programming Successful*.

No ensaio na placa, os interruptores SW5 e SW4 escolhem qual dos interruptores SW0 a SW3 aparece no LED LD0: o LED segue o interruptor selecionado, sendo indiferente o estado dos outros. Na Figura 4, SW5 = 0 e SW4 = 1 selecionam a entrada DataIn1; como SW1 está a 1, o LED LD0 está aceso.

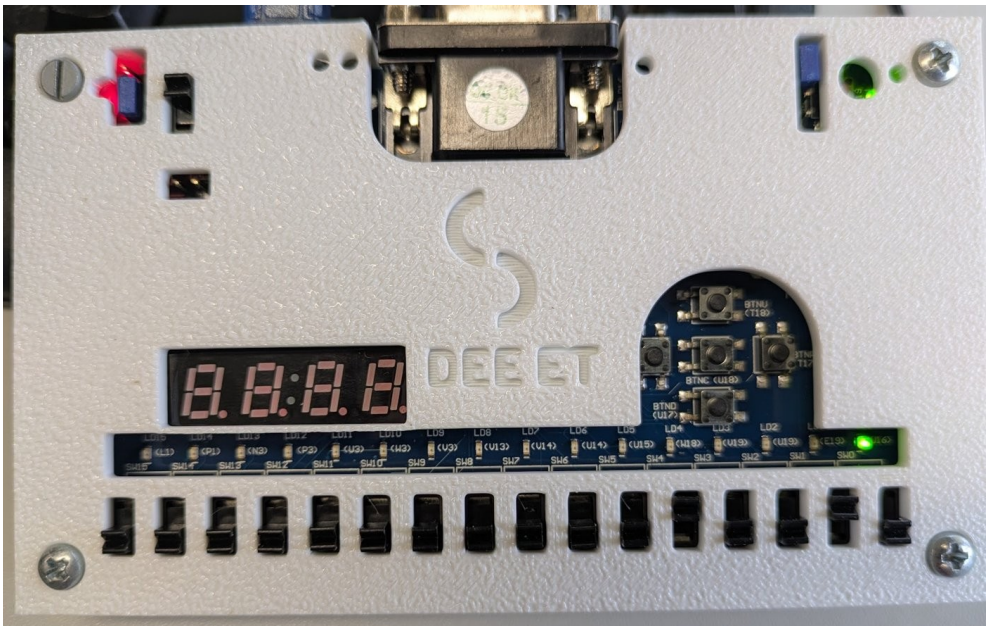


Figura 4: Ensaio do multiplexer 4x1 na placa Basys3: seleção 01 e SW1 = 1, LD0 aceso.

3 Descodificador 2x4 com enables individuais

Repetindo o procedimento anterior, desenhou-se um descodificador de 2 *bits* com um *enable* individual em cada saída (Fig. 26 do enunciado). A cada combinação das entradas Bit1 e Bit0 corresponde a ativação de uma única saída, desde que o respetivo *enable* esteja a 1; se o *enable* dessa saída estiver a 0, a saída fica a 0, e os *enables* das saídas não selecionadas não têm qualquer efeito:

$$Out0 = Bit1' \cdot Bit0' \cdot En0 \quad Out1 = Bit1' \cdot Bit0 \cdot En1 \quad Out2 = Bit1 \cdot Bit0' \cdot En2 \quad Out3 = Bit1 \cdot Bit0 \cdot En3$$

Este módulo é usado no controlador dos *displays* (secção 8) para escolher qual dos quatro *displays* está ativo em cada instante, e os *enables* servem para ligar ou desligar cada *display*.

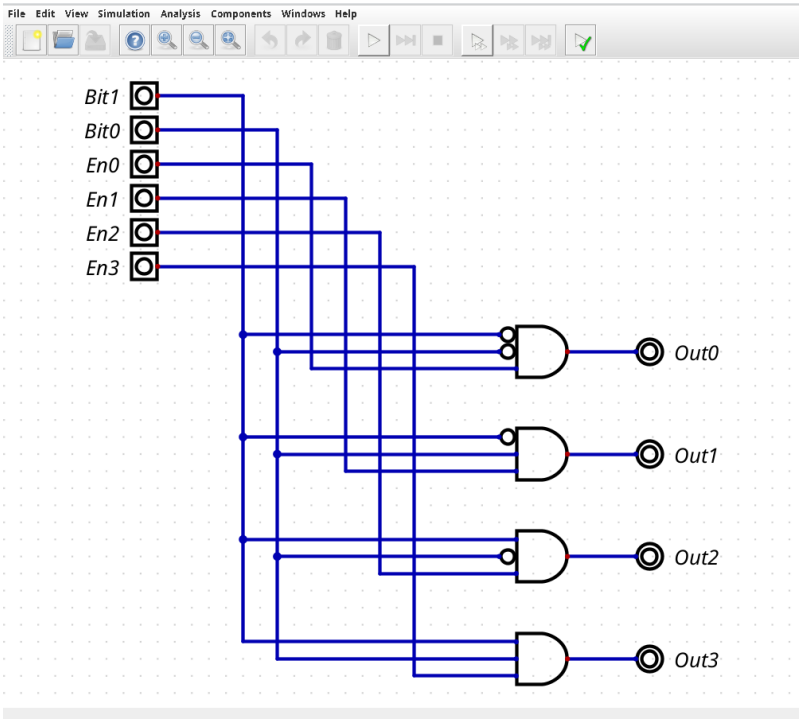


Figura 5: Esquema do descodificador 2x4 com enables individuais desenhado no Digital (`digital/descodificador2x4.dig`).

A Figura 6 mostra a simulação no Digital com Bit1Bit0 = 10 e os *enables* En1 e En2 a 1: só a saída Out2 fica ativa. O *enable* En1 também está a 1, mas, como a saída 1 não está selecionada, não tem efeito. A Tabela 4 resume o funcionamento observado nos testes.

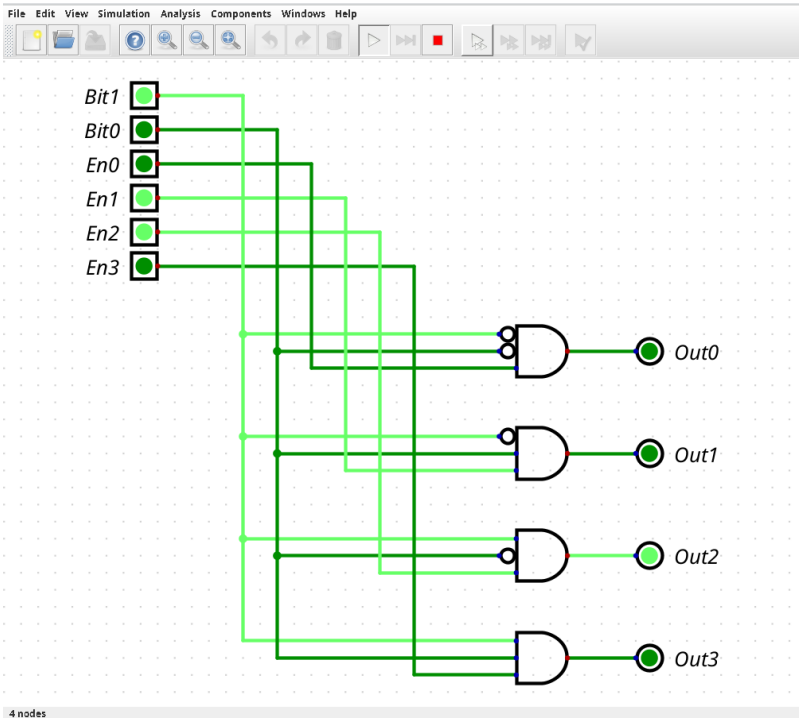


Figura 6: Simulação do descodificador 2x4 no Digital (Bit1 = 1, Bit0 = 0, En1 = En2 = 1): só Out2 fica a 1.

Tabela 4: Funcionamento do descodificador 2x4 com enables individuais (X = indiferente).

Bit1	Bit0	En3 En2 En1 En0	Out3 Out2 Out1 Out0
0	0	X X X 1	0 0 0 1
0	1	X X 1 X	0 0 1 0
1	0	X 1 X X	0 1 0 0
1	1	1 X X X	1 0 0 0
X	X	En*i* = 0 (*i* = código selecionado)	0 0 0 0

O código exportado pelo Digital (Listagem 4) volta a ser comportamental, com uma atribuição contínua por saída. No ficheiro de restrições (Listagem 5) as entradas de seleção ficaram nos interruptores SW15 e SW14, os *enables* em SW0 a SW3 e as saídas nos LEDs LD0 a LD3, de modo que cada *enable* fica imediatamente abaixo do LED da saída que controla.

Listagem 4: Código Verilog do descodificador 2x4 gerado pelo Digital (`descodificador2x4/descodificador2x4.v`)

```
/*
 * Generated by Digital. Don't modify this file!
 * Any changes will be lost if this file is regenerated.
 */

module descodificador2x4 (
  input Bit1,
  input Bit0,
  input En0,
  input En1,
  input En2,
  input En3,
  output Out0,
  output Out1,
  output Out2,
  output Out3
);
```

```
  assign Out0 = (~ Bit1 & ~ Bit0 & En0);
  assign Out1 = (~ Bit1 & Bit0 & En1);
  assign Out2 = (Bit1 & ~ Bit0 & En2);
  assign Out3 = (Bit1 & Bit0 & En3);
endmodule
```

Listagem 5: Ficheiro de restrições do descodificador 2x4 (`descodificador2x4/descodificador2x4.xdc`)

```
# Variáveis de entrada do descodificador 2x4 (interruptores SW15 e SW14)
set_property PACKAGE_PIN R2 [get_ports Bit1]
    set_property IOSTANDARD LVCMOS33 [get_ports Bit1]
set_property PACKAGE_PIN T1 [get_ports Bit0]
    set_property IOSTANDARD LVCMOS33 [get_ports Bit0]

# Enables individuais das saídas (interruptores SW0 a SW3)
set_property PACKAGE_PIN V17 [get_ports En0]
    set_property IOSTANDARD LVCMOS33 [get_ports En0]
set_property PACKAGE_PIN V16 [get_ports En1]
    set_property IOSTANDARD LVCMOS33 [get_ports En1]
set_property PACKAGE_PIN W16 [get_ports En2]
    set_property IOSTANDARD LVCMOS33 [get_ports En2]
set_property PACKAGE_PIN W17 [get_ports En3]
    set_property IOSTANDARD LVCMOS33 [get_ports En3]

# Variáveis de saída do descodificador 2x4 (LEDs LD0 a LD3)
set_property PACKAGE_PIN U16 [get_ports Out0]
    set_property IOSTANDARD LVCMOS33 [get_ports Out0]
set_property PACKAGE_PIN E19 [get_ports Out1]
    set_property IOSTANDARD LVCMOS33 [get_ports Out1]
set_property PACKAGE_PIN U19 [get_ports Out2]
    set_property IOSTANDARD LVCMOS33 [get_ports Out2]
set_property PACKAGE_PIN V19 [get_ports Out3]
    set_property IOSTANDARD LVCMOS33 [get_ports Out3]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* (Listagem 6) começa com todos os *enables* ativos e percorre os quatro códigos de entrada (acende uma saída de cada vez). Depois desativa os *enables* um a um, mostrando que a saída correspondente deixa de acender, e por fim verifica que um *enable* ativo numa saída **não selecionada** não acende nada (Bit1Bit0 = 11 só com En2 = 1, e Bit1Bit0 = 10 com todos os *enables* menos En2). A saída é comparada automaticamente com a expressão `(4'b0001 << B) & E`, que é exatamente a definição do descodificador.

Listagem 6: Testbench do descodificador 2x4 (`descodificador2x4/descodificador2x4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do descodificador 2x4 com enables individuais nas saídas
module descodificador2x4_teste1();

//   Declaração das entradas
     reg [1:0] B;          // Bit1, Bit0
     reg [3:0] E;          // En3..En0

//   Declaração das saídas
     wire [3:0] O;         // Out3..Out0

//   Instanciação da UUT e ligação das entradas e saídas
```

```
14      descodificador2x4 UUT (
15          .Bit1(B[1]),
16          .Bit0(B[0]),
17          .En0(E[0]),
18          .En1(E[1]),
19          .En2(E[2]),
20          .En3(E[3]),
21          .Out0(O[0]),
22          .Out1(O[1]),
23          .Out2(O[2]),
24          .Out3(O[3])
25      );
26
27  //   Declaração de vars auxiliares
28      integer n;            // Contador para o ciclo for
29      integer erros = 0;    // Número de saídas diferentes do esperado
30
31  //   Verificação automática: só a saída B pode estar ativa, e só se En[B]=1
32      always @(B or E) begin
33          #1;
34          if (O !== ((4'b0001 << B) & E)) begin
35              $display("ERRO t=%0t: B=%b E=%b O=%b (esperado %b)", $time, B, E, O, (4'b0001 << B) &
    E);
36              erros = erros + 1;
37          end
38      end
39
40  //   Definição do valor das entradas ao longo do tempo
41      initial begin
42          B = 2'b00;        // Valor inicial das vars de entrada
43          E = 4'b0000;      // Todas as saídas desativadas (O=0000)
44
45          #200;             // Aguardar 200ns para simular o tempo
46                            // das configurações iniciais da FPGA
47
48          for (n=0; n<16; n=n+1)
49          begin
50              #50;                          // Passam 50ns a cada ciclo de n
51              case (n)
52                   0: E = 4'b1111;          // Ativa todos os enables (O=0001)
53                   1: B = 2'b01;            // Descodifica 1 (O=0010)
54                   2: B = 2'b10;            // Descodifica 2 (O=0100)
55                   3: B = 2'b11;            // Descodifica 3 (O=1000)
56                   4: E = 4'b0111;          // Desativa En3: saída 3 apaga (O=0000)
57                   5: B = 2'b10;            // Descodifica 2 (O=0100)
58                   6: E = 4'b0011;          // Desativa En2 (O=0000)
59                   7: B = 2'b01;            // Descodifica 1 (O=0010)
60                   8: E = 4'b0001;          // Desativa En1 (O=0000)
61                   9: B = 2'b00;            // Descodifica 0 (O=0001)
62                  10: E = 4'b1110;          // Só En0 desativado (O=0000)
63                  11: B = 2'b11;            // Descodifica 3 (O=1000)
64                  12: E = 4'b0100;          // Só En2 ativo, mas B=3 (O=0000)
65                  13: B = 2'b10;            // Descodifica 2 (O=0100)
66                  14: E = 4'b1011;          // Todos menos En2, B=2 (O=0000)
67                  15: E = 4'b0000;          // Desativa tudo (O=0000)
68              endcase
69          end
70          #50;
71          if (erros == 0) $display("Simulacao OK: descodificador2x4 sem erros");
72          else            $display("Simulacao FALHOU: %0d erros", erros);
73          $finish;
74      end
```

```
75  endmodule
```

Na Figura 7 observa-se que, entre 250 e 400 ns, a saída ativa percorre O0, O1, O2 e O3 à medida que B passa de 0 a 3; a partir de 450 ns, sempre que o *enable* da saída selecionada é desligado, a saída 0 passa a 0000; entre 850 e 1000 ns, os *enables* das saídas não selecionadas não têm efeito. O simulador indicou `Simulacao OK: descodificador2x4 sem erros`. O *bitstream* foi gerado sem erros (2 LUT).

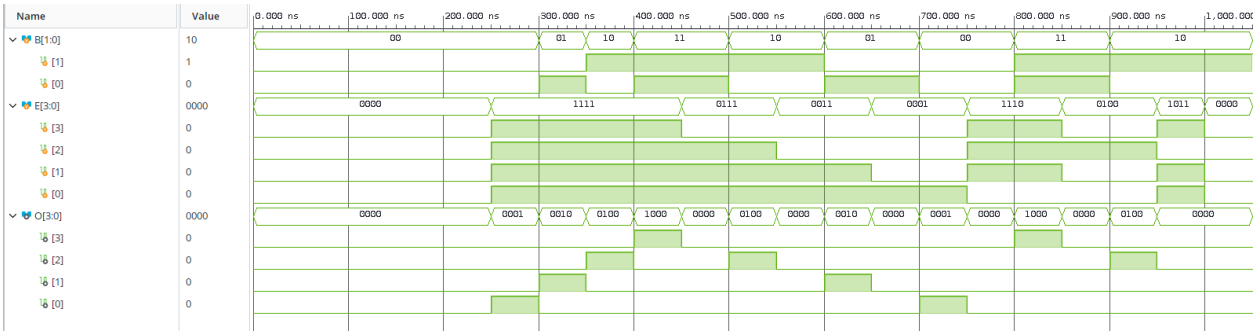
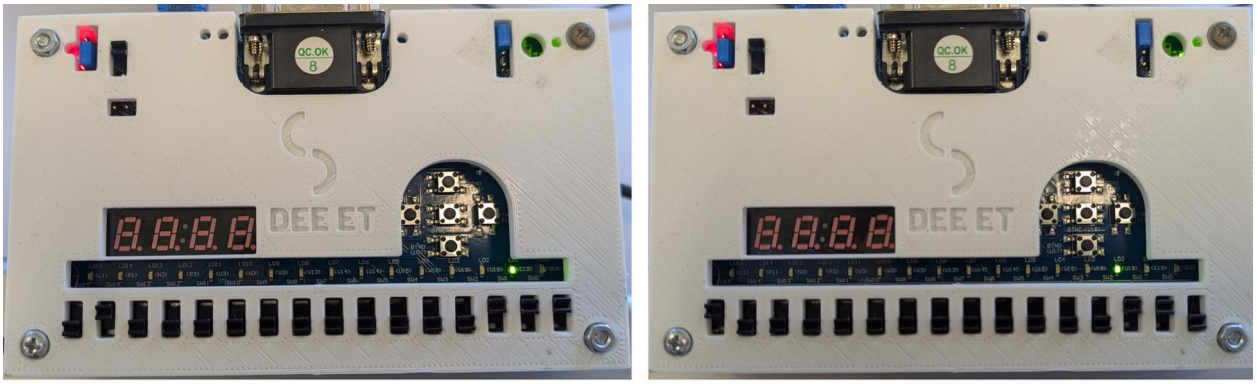


Figura 7: Simulação comportamental do descodificador 2x4 no Vivado.

No ensaio na placa (Figura 8), os interruptores SW15 e SW14 escolhem a saída e SW0 a SW3 são os *enables*. Em (a), SW15 SW14 = 01 seleciona a saída 1 e, como o *enable* SW1 está ligado, só o LED LD1 acende; em (b), SW15 SW14 = 10 seleciona a saída 2, com SW2 ligado, e só o LED LD2 acende. Em ambos os casos, o *enable* ligado de uma saída não selecionada não acende o respetivo LED.



(a) **(b)**

Figura 8: Ensaio do descodificador 2x4 na placa Basys3: (a) saída 1 selecionada, LD1 aceso; (b) saída 2 selecionada, LD2 aceso.

4 Multiplexer 16x4

O *multiplexer* 16x4 tem quatro entradas de 4 *bits* (DataIn0 a DataIn3), duas variáveis de seleção (`Sel[1:0]`) e uma saída de 4 *bits* (`DataOut`). O módulo foi criado diretamente no Vivado (Fig. 27 do enunciado) e, em vez de descrever a sua função, foi construído por **hierarquia de módulos**: instanciam-se quatro *multiplexers* 4x1 (`mux0` a `mux3`), em que o *multiplexer* `mux`*i* trata o *bit i* das quatro entradas e todos partilham as variáveis de seleção (Listagem 7). Trata-se, portanto, de uma descrição **estrutural**. A Figura 9 mostra a hierarquia resultante no Vivado, igual à da Fig. 29 do enunciado.

Listagem 7: Multiplexer 16x4 construído com quatro multiplexers 4x1 (`multiplexer16x4/multiplexer16x4.v`)

```
1  `timescale 1ns / 1ps
2
3  // Multiplexer 16x4: 4 entradas de 4 bits, selecionadas por Sel[1:0].
4  // Construído estruturalmente com 4 multiplexers 4x1, um por cada bit.
5  module multiplexer16x4(
6      input [3:0] DataIn0,
7      input [3:0] DataIn1,
8      input [3:0] DataIn2,
9      input [3:0] DataIn3,
10     output [3:0] DataOut,
11     input [1:0] Sel
12     );
13
14 //   Instanciação do mux4x1 do bit 0 e ligação das entradas e saídas
15     multiplexer4x1 mux0 (
16         .Sel1(Sel[1]),
17         .Sel0(Sel[0]),
18         .DataIn0(DataIn0[0]),
19         .DataIn1(DataIn1[0]),
20         .DataIn2(DataIn2[0]),
21         .DataIn3(DataIn3[0]),
22         .DataOut(DataOut[0])
23     );
24
25 //   Instanciação do mux4x1 do bit 1 e ligação das entradas e saídas
26     multiplexer4x1 mux1 (
27         .Sel1(Sel[1]),
28         .Sel0(Sel[0]),
29         .DataIn0(DataIn0[1]),
30         .DataIn1(DataIn1[1]),
31         .DataIn2(DataIn2[1]),
32         .DataIn3(DataIn3[1]),
33         .DataOut(DataOut[1])
34     );
35
36 //   Instanciação do mux4x1 do bit 2 e ligação das entradas e saídas
37     multiplexer4x1 mux2 (
38         .Sel1(Sel[1]),
39         .Sel0(Sel[0]),
40         .DataIn0(DataIn0[2]),
41         .DataIn1(DataIn1[2]),
42         .DataIn2(DataIn2[2]),
43         .DataIn3(DataIn3[2]),
44         .DataOut(DataOut[2])
45     );
46
47 //   Instanciação do mux4x1 do bit 3 e ligação das entradas e saídas
48     multiplexer4x1 mux3 (
49         .Sel1(Sel[1]),
50         .Sel0(Sel[0]),
51         .DataIn0(DataIn0[3]),
```

```
52          .DataIn1(DataIn1[3]),
53          .DataIn2(DataIn2[3]),
54          .DataIn3(DataIn3[3]),
55          .DataOut(DataOut[3])
56      );
57
58  endmodule
```

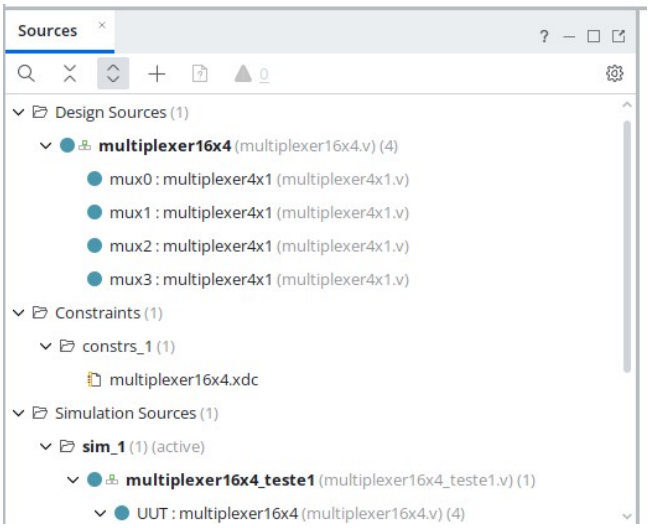


Figura 9: Hierarquia de módulos do multiplexer 16x4 na janela Sources do Vivado.

São precisos 16 *bits* de dados, pelo que se usaram os 16 interruptores da placa para as entradas; a seleção ficou em duas teclas e a saída nos LEDs LD3 a LD0 (Tabela 5 e Listagem 8).

Tabela 5: Ligação dos sinais do multiplexer 16x4 à Basys3.

Sinal	Elemento da Basys3
DataIn0[3:0]	Interruptores SW3 a SW0
DataIn1[3:0]	Interruptores SW7 a SW4
DataIn2[3:0]	Interruptores SW11 a SW8
DataIn3[3:0]	Interruptores SW15 a SW12
Sel[1] / Sel[0]	Teclas BTNL / BTNR (premida = 1)
DataOut[3:0]	LEDs LD3 a LD0

Listagem 8: Ficheiro de restrições do multiplexer 16x4 (`multiplexer16x4/multiplexer16x4.xdc`)

```
1  # Variáveis de seleção do multiplexer 16x4 (teclas BTNL e BTNR)
2  set_property PACKAGE_PIN W19 [get_ports {Sel[1]}]
3      set_property IOSTANDARD LVCMOS33 [get_ports {Sel[1]}]
4  set_property PACKAGE_PIN T17 [get_ports {Sel[0]}]
5      set_property IOSTANDARD LVCMOS33 [get_ports {Sel[0]}]
6
7  # Entrada DataIn0 do multiplexer 16x4 (interruptores SW3 a SW0)
8  set_property PACKAGE_PIN W17 [get_ports {DataIn0[3]}]
9  set_property PACKAGE_PIN W16 [get_ports {DataIn0[2]}]
10 set_property PACKAGE_PIN V16 [get_ports {DataIn0[1]}]
11 set_property PACKAGE_PIN V17 [get_ports {DataIn0[0]}]
12     set_property IOSTANDARD LVCMOS33 [get_ports {DataIn0[*]}]
13
14 # Entrada DataIn1 do multiplexer 16x4 (interruptores SW7 a SW4)
15 set_property PACKAGE_PIN W13 [get_ports {DataIn1[3]}]
16 set_property PACKAGE_PIN W14 [get_ports {DataIn1[2]}]
17 set_property PACKAGE_PIN V15 [get_ports {DataIn1[1]}]
```

```
set_property PACKAGE_PIN W15 [get_ports {DataIn1[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataIn1[*]}]

# Entrada DataIn2 do multiplexer 16x4 (interruptores SW11 a SW8)
set_property PACKAGE_PIN R3 [get_ports {DataIn2[3]}]
set_property PACKAGE_PIN T2 [get_ports {DataIn2[2]}]
set_property PACKAGE_PIN T3 [get_ports {DataIn2[1]}]
set_property PACKAGE_PIN V2 [get_ports {DataIn2[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataIn2[*]}]

# Entrada DataIn3 do multiplexer 16x4 (interruptores SW15 a SW12)
set_property PACKAGE_PIN R2 [get_ports {DataIn3[3]}]
set_property PACKAGE_PIN T1 [get_ports {DataIn3[2]}]
set_property PACKAGE_PIN U1 [get_ports {DataIn3[1]}]
set_property PACKAGE_PIN W2 [get_ports {DataIn3[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataIn3[*]}]

# Saída do multiplexer 16x4 (LEDs LD3 a LD0)
set_property PACKAGE_PIN V19 [get_ports {DataOut[3]}]
set_property PACKAGE_PIN U19 [get_ports {DataOut[2]}]
set_property PACKAGE_PIN E19 [get_ports {DataOut[1]}]
set_property PACKAGE_PIN U16 [get_ports {DataOut[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataOut[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* (Listagem 9) coloca valores diferentes nas quatro entradas (A, 5, C e 3, em hexadecimal) e percorre as quatro seleções; depois altera uma entrada **não selecionada** (a saída não muda) e a entrada **selecionada** (a saída acompanha-a); por fim, aplica os valores 1, 2, 4 e 8 numa entrada, para confirmar que cada um dos quatro *multiplexers* 4x1 está bem ligado ao *bit* correspondente. A saída é comparada continuamente com a entrada selecionada.

Listagem 9: Testbench do multiplexer 16x4 (`multiplexer16x4/multiplexer16x4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do multiplexer 16x4
module multiplexer16x4_teste1();

//   Declaração das entradas
     reg [1:0] S;
     reg [3:0] D0, D1, D2, D3;

//   Declaração da saída
     wire [3:0] Z;

//   Instanciação da UUT e ligação das entradas e saídas
     multiplexer16x4 UUT (
         .DataIn0(D0),
         .DataIn1(D1),
         .DataIn2(D2),
         .DataIn3(D3),
         .DataOut(Z),
         .Sel(S)
     );

//   Declaração de vars auxiliares
     integer n;            // Contador para o ciclo for
     integer erros = 0;    // Número de saídas diferentes do esperado
```

```
26      reg [3:0] esperado;
27
28  //  Verificação automática: a saída deve ser sempre igual à entrada selecionada
29      always @(S or D0 or D1 or D2 or D3) begin
30          #1;
31          case (S)
32              2'd0: esperado = D0;
33              2'd1: esperado = D1;
34              2'd2: esperado = D2;
35              2'd3: esperado = D3;
36          endcase
37          if (Z !== esperado) begin
38              $display("ERRO t=%0t: S=%d Z=%h (esperado %h)", $time, S, Z, esperado);
39              erros = erros + 1;
40          end
41      end
42
43  //  Definição do valor das entradas ao longo do tempo
44      initial begin
45          S  = 2'b00;       // Valor inicial das vars de seleção
46          D0 = 4'h0;        // Valor inicial das vars de entrada
47          D1 = 4'h0;
48          D2 = 4'h0;
49          D3 = 4'h0;
50
51          #200;             // Aguardar 200ns para simular o tempo
52                            // das configurações iniciais da FPGA
53
54          for (n=0; n<14; n=n+1)
55          begin
56              #50;                          // Passam 50ns a cada ciclo de n
57              case (n)
58                   0: begin                 // Valores distintos nas 4 entradas
59                          D0 = 4'hA;        // (Z=A)
60                          D1 = 4'h5;
61                          D2 = 4'hC;
62                          D3 = 4'h3;
63                      end
64                   1: S = 2'b01;            // Seleciona DataIn1 (Z=5)
65                   2: S = 2'b10;            // Seleciona DataIn2 (Z=C)
66                   3: S = 2'b11;            // Seleciona DataIn3 (Z=3)
67                   4: D0 = 4'hF;            // Muda entrada não selecionada (Z=3)
68                   5: D3 = 4'h9;            // Muda entrada selecionada (Z=9)
69                   6: S = 2'b00;            // Seleciona DataIn0 (Z=F)
70                   7: D0 = 4'h1;            // Testa cada bit isolado (Z=1)
71                   8: D0 = 4'h2;            // (Z=2)
72                   9: D0 = 4'h4;            // (Z=4)
73                  10: D0 = 4'h8;            // (Z=8)
74                  11: S = 2'b10;            // Seleciona DataIn2 (Z=C)
75                  12: D2 = 4'h0;            // (Z=0)
76                  13: S = 2'b01;            // Seleciona DataIn1 (Z=5)
77              endcase
78          end
79          #50;
80          if (erros == 0) $display("Simulacao OK: multiplexer16x4 sem erros");
81          else            $display("Simulacao FALHOU: %0d erros", erros);
82          $finish;
83      end
84  endmodule
```

Na Figura 10 (barramentos em hexadecimal) a saída Z apresenta sucessivamente A, 5, C e 3 entre 250 e 450 ns; mantém-se em 3 quando D0 muda para F (D0 não está selecionada) e passa a 9 quando a entrada selecionada D3 muda; depois apresenta F, 1, 2, 4, 8, C, 0 e 5, sempre igual à entrada selecionada. O resultado da simulação foi `Simulacao OK: multiplexer16x4 sem erros` e o *bitstream* foi gerado sem erros (4 LUT: uma por *bit* da saída, cada uma com 6 entradas).

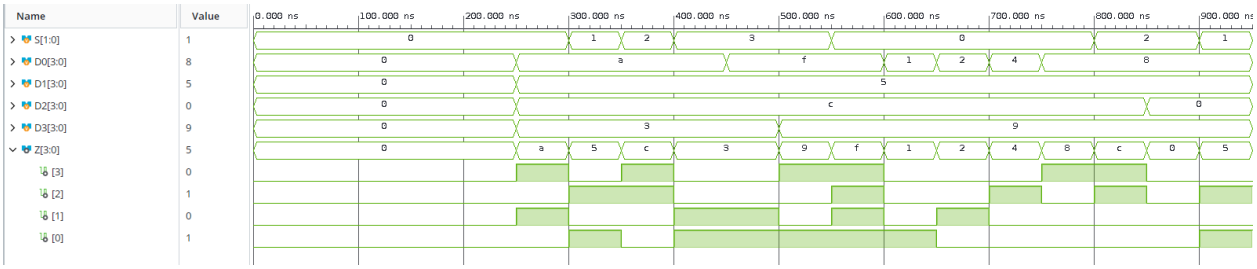
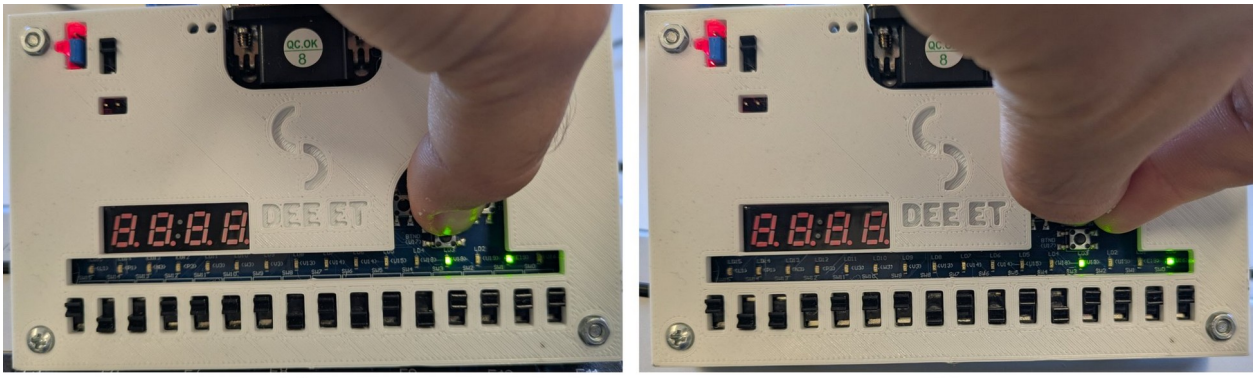


Figura 10: Simulação comportamental do multiplexer 16x4 no Vivado.

No ensaio na placa (Figura 11), cada grupo de quatro interruptores tem um valor diferente e as teclas BTNL e BTNR escolhem o grupo que aparece nos LEDs LD3 a LD0. Em (a), com a tecla BTNR premida, os LEDs mostram 1010, o valor de SW7 a SW4 (entrada DataIn1); em (b), com as duas teclas premidas, mostram 1001, o valor de SW15 a SW12 (entrada DataIn3).



(a) **(b)**

Figura 11: Ensaio do multiplexer 16x4 na placa Basys3: (a) BTNR premida, entrada DataIn1; (b) BTNL e BTNR premidas, entrada DataIn3.

5 Transcodificador BCD para display de 7 segmentos

O transcodificador `Transcod7Seg` recebe o barramento `Numero[3:0]` e produz o barramento `Segmentos[6:0]`, em que o *bit* mais significativo (índice 6) corresponde ao segmento *a* e o menos significativo (índice 0) ao segmento *g*. As saídas são **ativas a Low**: um *bit* a 0 acende o respetivo segmento. O ponto decimal não é tratado neste módulo.

Para as entradas 0 a 9 foram usadas as configurações dos algarismos da Fig. 30 do enunciado. Para as entradas 10 a 15, que não são BCD, foram criados símbolos originais (Tabela 6 e Figura 12): as iniciais dos dois elementos do grupo, **r** e **d**, as letras **L**, **E** e **C** (de LEEC) e um traço. As letras foram escolhidas de forma a não se confundirem com nenhum algarismo: o "r" usa só os segmentos *e* e *g*, o "d" é minúsculo (o "D" maiúsculo seria igual ao 0) e o "C" não tem o segmento *g*, ao contrário do 6. Os 16 símbolos são todos diferentes entre si, o que é verificado no *testbench*.

Tabela 6: Configuração dos segmentos para cada valor de Numero (0 = segmento aceso).

Numero	Segmentos[6:0] (a b c d e f g)	Segmentos acesos	Símbolo
0	0000001	a b c d e f	0
1	1001111	b c	1
2	0010010	a b d e g	2
3	0000110	a b c d g	3
4	1001100	b c f g	4
5	0100100	a c d f g	5
6	0100000	a c d e f g	6
7	0001111	a b c	7
8	0000000	a b c d e f g	8
9	0000100	a b c d f g	9
10	1111010	e g	r
11	1000010	b c d e g	d
12	1110001	d e f	L
13	0110000	a d e f g	E
14	0110001	a d e f	C
15	1111110	g	-

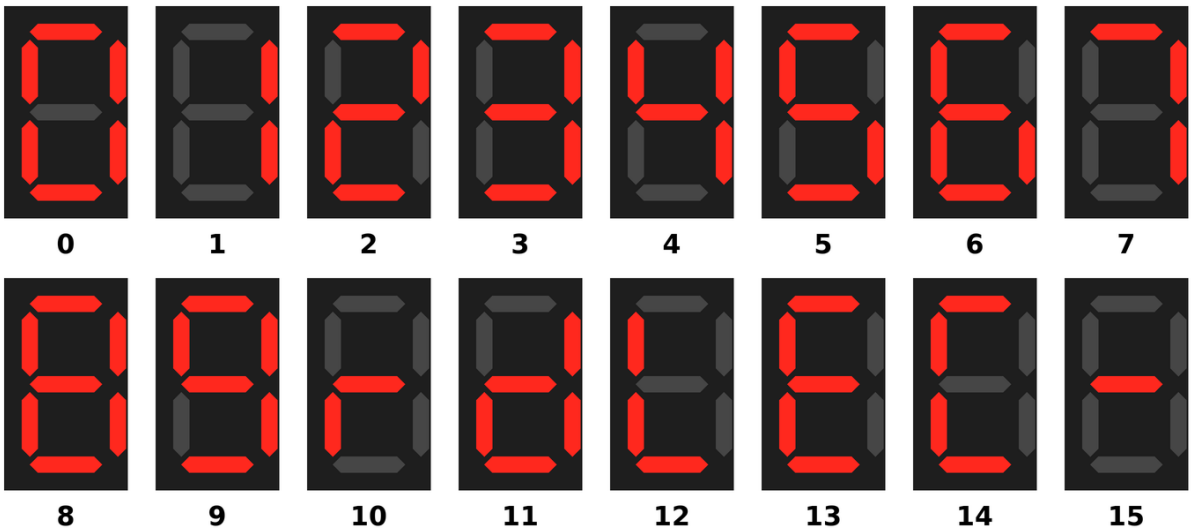


Figura 12: Os 16 símbolos do transcodificador, desenhados a partir dos valores de `transcod7seg.v` (a vermelho os segmentos acesos).

O módulo foi adaptado do transcodificador `Transcod7Seg` dos exemplos Verilog disponibilizados no Moodle, que tem erros introduzidos intencionalmente; esses erros e as correções feitas são analisados na secção 9.2. O código final está na Listagem 10. Por ser um circuito combinatório, a saída é declarada `output reg` e atribuída num bloco `always @(*)` com atribuições bloqueantes (=); como os 16 valores possíveis da entrada estão todos no `case`, o `default` nunca é usado, mas mantém-se por segurança.

Listagem 10: Transcodificador para display de 7 segmentos (adaptado do exemplo do Moodle) (`transcod7seg/transcod7seg.v`)

```
1  `timescale 1ns / 1ps
2
3  // Transcodificador de BCD (e símbolos 10 a 15) para display de 7 segmentos.
4  // Segmentos[6] = a ... Segmentos[0] = g, ativos a Low (0 = segmento aceso).
5  // O ponto decimal não é tratado neste módulo.
6  // Adaptado do módulo Transcod7Seg dos exemplos Verilog do Moodle,
7  // com os erros corrigidos e símbolos 10 a 15 originais (ver relatório, secção 9.2).
8  module Transcod7Seg(
9      input [3:0] Numero,
10     output reg [6:0] Segmentos
11     );
12
13      always @(*)
14          case (Numero)               //  abcdefg
15              4'd0:  Segmentos = 7'b0000001;   // 0
16              4'd1:  Segmentos = 7'b1001111;   // 1
17              4'd2:  Segmentos = 7'b0010010;   // 2
18              4'd3:  Segmentos = 7'b0000110;   // 3
19              4'd4:  Segmentos = 7'b1001100;   // 4
20              4'd5:  Segmentos = 7'b0100100;   // 5
21              4'd6:  Segmentos = 7'b0100000;   // 6
22              4'd7:  Segmentos = 7'b0001111;   // 7
23              4'd8:  Segmentos = 7'b0000000;   // 8
24              4'd9:  Segmentos = 7'b0000100;   // 9
25              // Símbolos originais nas combinações não BCD: iniciais do grupo (r, d),
26              // LEC (de LEEC) e um traço: "rdLEC-"
27              4'd10: Segmentos = 7'b1111010;   // r (e g)
28              4'd11: Segmentos = 7'b1000010;   // d (b c d e g)
29              4'd12: Segmentos = 7'b1110001;   // L (d e f)
30              4'd13: Segmentos = 7'b0110000;   // E (a d e f g)
31              4'd14: Segmentos = 7'b0110001;   // C (a d e f)
32              4'd15: Segmentos = 7'b1111110;   // - (g)
33              default: Segmentos = 7'b1111111; // tudo apagado (nunca usado: os 16 casos estão
    cobertos)
34          endcase
35
36 endmodule
```

O *testbench* (Listagem 11) guarda uma tabela de referência escrita em lógica positiva (segmentos **acesos**), fácil de comparar com a Fig. 30, e compara a saída com a negação dessa tabela. Verifica ainda que nenhum segmento fica indefinido (X ou Z), que os 16 símbolos são todos diferentes e que a saída só depende do valor atual da entrada: para isso, depois de percorrer os 16 valores por ordem crescente, repete-os duas vezes por ordem aleatória (48 verificações no total).

Listagem 11: Testbench do transcodificador (`transcod7seg/transcod7seg_teste1.v`)

```
1  `timescale 1ns / 1ps
2
3  // Testbench do transcodificador para 7 segmentos.
4  // Percorre os 16 valores de Numero e verifica, para cada um:
5  //   - que a saída é igual à tabela de referência (Fig. 30 e símbolos 10 a 15);
6  //   - que nenhum bit da saída fica indefinido (X ou Z);
```

```
//   - que os 16 símbolos são todos diferentes entre si (nenhum é ambíguo no display).
// Depois repete os 16 valores por ordem aleatória, para garantir que a saída
// só depende do valor atual e não do valor anterior (circuito combinatório).
module transcod7seg_teste1();

//   Declaração das entradas
     reg [3:0] N;

//   Declaração da saída (a..g, ativos a Low)
     wire [6:0] S;

//   Instanciação da UUT e ligação das entradas e saídas
     Transcod7Seg UUT (
         .Numero(N),
         .Segmentos(S)
     );

//   Tabela de referência, escrita com os segmentos ACESOS (ativo a High) para
//   ser fácil de comparar com a Fig. 30; a saída esperada é a sua negação.
     reg [6:0] acesos [0:15];
     initial begin           //  abcdefg
         acesos[0]  = 7'b1111110;  acesos[1]  = 7'b0110000;
         acesos[2]  = 7'b1101101;  acesos[3]  = 7'b1111001;
         acesos[4]  = 7'b0110011;  acesos[5]  = 7'b1011011;
         acesos[6]  = 7'b1011111;  acesos[7]  = 7'b1110000;
         acesos[8]  = 7'b1111111;  acesos[9]  = 7'b1111011;
         acesos[10] = 7'b0000101;  acesos[11] = 7'b0111101;  // r  d
         acesos[12] = 7'b0001110;  acesos[13] = 7'b1001111;  // L  E
         acesos[14] = 7'b1001110;  acesos[15] = 7'b0000001;  // C  -
     end

//   Declaração de vars auxiliares
     integer n, m, k, erros = 0, verificacoes = 0;
     integer semente = 7;
     reg [6:0] obtido [0:15];
     reg [3:0] ordem [0:15];
     reg [3:0] tmp;

     task verifica(input [3:0] valor);
         begin
             verificacoes = verificacoes + 1;
             if (^S === 1'bx) begin
                 $display("ERRO: Numero=%0d tem segmentos indefinidos (%b)", valor, S);
                 erros = erros + 1;
             end
             else if (S !== ~acesos[valor]) begin
                 $display("ERRO: Numero=%0d Segmentos=%b (esperado %b)", valor, S, ~acesos[valor]);
                 erros = erros + 1;
             end
         end
     endtask

//   Definição do valor das entradas ao longo do tempo
     initial begin
         N = 4'h0;
         #200;                   // Aguardar as configurações iniciais da FPGA

         // 1) Os 16 valores por ordem crescente
         for (n=0; n<16; n=n+1)
         begin
             N = n;
             #50;                // Passam 50ns em cada valor
```

```
            verifica(n);
            obtido[n] = S;
        end

        // 2) Os 16 símbolos têm de ser todos diferentes
        for (n=0; n<16; n=n+1)
            for (m=n+1; m<16; m=m+1)
                if (obtido[n] === obtido[m]) begin
                    $display("ERRO: Numero=%0d e Numero=%0d mostram o mesmo simbolo", n, m);
                    erros = erros + 1;
                end

        // 3) Os 16 valores por ordem aleatória (baralhados), duas vezes
        for (k=0; k<2; k=k+1) begin
            for (n=0; n<16; n=n+1) ordem[n] = n;
            for (n=15; n>0; n=n-1) begin
                m = {$random(semente)} % (n+1);
                tmp = ordem[n]; ordem[n] = ordem[m]; ordem[m] = tmp;
            end
            for (n=0; n<16; n=n+1) begin
                N = ordem[n];
                #20 verifica(ordem[n]);
            end
        end

        if (erros == 0) $display("Simulacao OK: Transcod7Seg sem erros (%0d verificacoes)",
    verificacoes);
        else            $display("Simulacao FALHOU: Transcod7Seg com %0d erros", erros);
        $finish;
    end
endmodule
```

A Figura 13 mostra a primeira parte da simulação (de 200 a 1000 ns), em que `N` percorre os valores de 0 a 15 de 50 em 50 ns. Para cada valor, os *bits* de `S` coincidem com a coluna Segmentos[6:0] da Tabela 6: por exemplo, para N = 8 todos os *bits* estão a 0 (todos os segmentos acesos) e para N = 15 só `S[0]` (segmento *g*) está a 0; entre 700 e 1000 ns veem-se os símbolos originais r, d, L, E, C e -. O simulador indicou `Simulacao OK: Transcod7Seg sem erros (48 verificacoes)`.

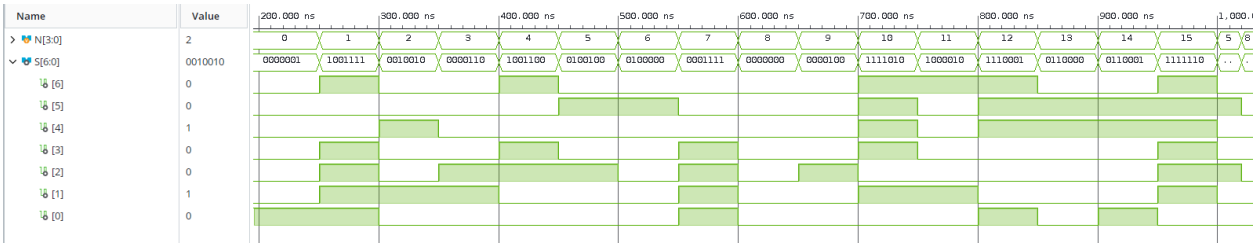


Figura 13: Simulação comportamental do transcodificador no Vivado (200 a 1000 ns).

5.1 Ensaio do transcodificador na placa

Para ensaiar o transcodificador isoladamente na Basys3, criou-se o módulo de topo `displayBCD` (Listagem 12): o valor a apresentar vem dos interruptores SW3 a SW0 e os interruptores SW15 a SW12 escolhem quais dos quatro *displays* acendem. Como os ânodos comuns são ativos a *Low*, o módulo nega o valor desses interruptores; o ponto decimal fica sempre apagado (nível 1). Os pinos estão na Tabela 7 e no ficheiro de restrições da Listagem 13.

Listagem 12: Circuito de ensaio do transcodificador (`transcod7seg/displayBCD.v`)

```
`timescale 1ns / 1ps

```

```
// Circuito de ensaio do transcodificador na Basys3.
// SW3..SW0 escolhem o símbolo e SW15..SW12 escolhem quais displays acendem.
// Os ânodos são comuns e ativos a Low, por isso inverte-se o valor dos interruptores.
module displayBCD(
    input [3:0] Numero,
    input [3:0] Liga,
    output [6:0] Segmentos,
    output DotPoint,
    output [3:0] Anodos
    );

     Transcod7Seg transcod (
         .Numero(Numero),
         .Segmentos(Segmentos)
     );

     assign Anodos = ~Liga;
     assign DotPoint = 1'b1;       // ponto decimal apagado

endmodule
```

Tabela 7: Ligação dos sinais do circuito `displayBCD` à Basys3.

Sinal	Pinos da FPGA	Elemento da Basys3
Numero[3:0]	W17 W16 V16 V17	Interruptores SW3 a SW0
Liga[3:0]	R2 T1 U1 W2	Interruptores SW15 a SW12
Segmentos[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a Low)
DotPoint	V7	Ponto decimal DP (ativo a Low)
Anodos[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a Low)

Listagem 13: Ficheiro de restrições do circuito `displayBCD` (`transcod7seg/displayBCD.xdc`)

```
# Ensaio do transcodificador para 7 segmentos (módulo de topo displayBCD)

# Número a apresentar (interruptores SW3 a SW0)
set_property PACKAGE_PIN W17 [get_ports {Numero[3]}]
set_property PACKAGE_PIN W16 [get_ports {Numero[2]}]
set_property PACKAGE_PIN V16 [get_ports {Numero[1]}]
set_property PACKAGE_PIN V17 [get_ports {Numero[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Numero[*]}]

# Displays a ligar (interruptores SW15 a SW12, um por display)
set_property PACKAGE_PIN R2 [get_ports {Liga[3]}]
set_property PACKAGE_PIN T1 [get_ports {Liga[2]}]
set_property PACKAGE_PIN U1 [get_ports {Liga[1]}]
set_property PACKAGE_PIN W2 [get_ports {Liga[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Liga[*]}]

# Segmentos a..g (Segmentos[6] = a ... Segmentos[0] = g), ativos a Low
set_property PACKAGE_PIN W7 [get_ports {Segmentos[6]}]
set_property PACKAGE_PIN W6 [get_ports {Segmentos[5]}]
set_property PACKAGE_PIN U8 [get_ports {Segmentos[4]}]
set_property PACKAGE_PIN V8 [get_ports {Segmentos[3]}]
set_property PACKAGE_PIN U5 [get_ports {Segmentos[2]}]
set_property PACKAGE_PIN V5 [get_ports {Segmentos[1]}]
set_property PACKAGE_PIN U7 [get_ports {Segmentos[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Segmentos[*]}]

# Ponto decimal
set_property PACKAGE_PIN V7 [get_ports DotPoint]
```

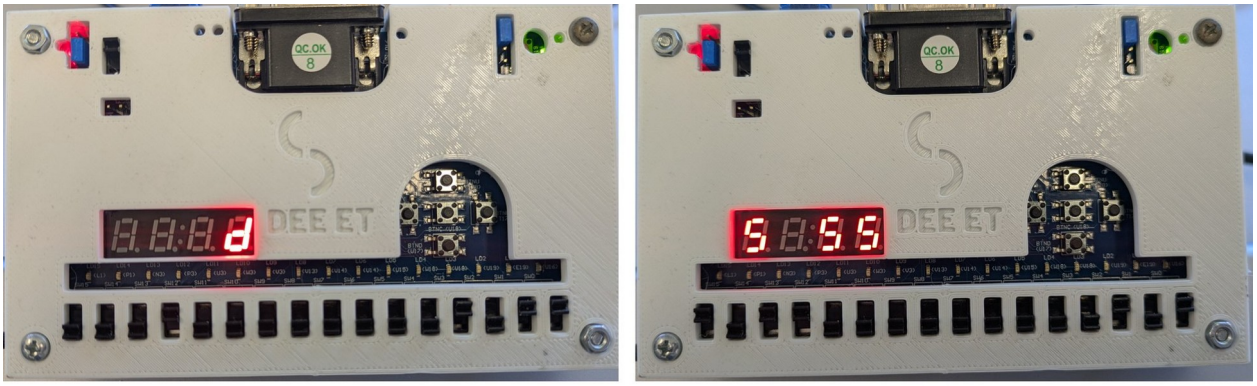
```
    set_property IOSTANDARD LVCMOS33 [get_ports DotPoint]

# Ânodos comuns dos 4 displays (ativos a Low); Anodos[0] é o display da direita
set_property PACKAGE_PIN U2 [get_ports {Anodos[0]}]
set_property PACKAGE_PIN U4 [get_ports {Anodos[1]}]
set_property PACKAGE_PIN V4 [get_ports {Anodos[2]}]
set_property PACKAGE_PIN W4 [get_ports {Anodos[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anodos[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *bitstream* foi gerado sem erros e o circuito ocupa 8 LUT.

Na Figura 14 mostram-se dois ensaios na placa. Em (a), só SW12 está ligado e SW3 a SW0 = 1011: o *display* da direita apresenta o símbolo original "d" (valor 11). Em (b), com SW15, SW13 e SW12 ligados e SW3 a SW0 = 0101, três *displays* apresentam o algarismo 5 e o segundo a contar da esquerda fica apagado, o que confirma a negação dos ânodos.



(a) **(b)**

Figura 14: Ensaio do transcodificador na placa Basys3: (a) símbolo "d" no display da direita; (b) algarismo 5 em três displays.

6 Registo de 4 bits com clear e enable

O registo `Registo4` foi construído com quatro instâncias da primitiva **FDCE** da Xilinx, uma por *bit* (Listagem 14). A FDCE é um *flip-flop* D sensível ao flanco ascendente do relógio (`C`), com *clock enable* (`CE`) e *clear* assíncrono (`CLR`, ativo a *High*): quando `CLR = 1` a saída passa imediatamente a 0, sem esperar pelo relógio, e o *clear* tem prioridade sobre a carga; com `CLR = 0`, a saída copia a entrada `D` em cada flanco ascendente do relógio se `CE = 1` e mantém o valor se `CE = 0`. O parâmetro `INIT` define o valor inicial do *flip-flop* após a configuração da FPGA (neste caso, 0). As portas do módulo seguem a Fig. 31 do enunciado (`Data`, `CE`, `Clk`, `CLR` e `DataOut`). A instanciação segue o modelo disponível no Vivado em Tools > Language Templates (Verilog > Device Primitive Instantiation > Artix-7 > Register > FDCE).

Listagem 14: Registo de 4 bits com quatro primitivas FDCE (`registo4/registo4.v`)

```
1  `timescale 1ns / 1ps
2
3  // Registo de 4 bits com clear assíncrono e clock enable.
4  // Feito com 4 primitivas FDCE (flip-flop D com CLR assíncrono e CE),
5  // sensíveis ao flanco ascendente do relógio.
6  module Registo4(
7      input [3:0] Data,
8      input CE,
9      input Clk,
10     input CLR,
11     output [3:0] DataOut
12     );
13
14 //   Instanciação dos 4 flip-flops FDCE, um por cada bit
15      FDCE #(.INIT(1'b0)) ff0 (.Q(DataOut[0]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[0]));
16      FDCE #(.INIT(1'b0)) ff1 (.Q(DataOut[1]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[1]));
17      FDCE #(.INIT(1'b0)) ff2 (.Q(DataOut[2]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[2]));
18      FDCE #(.INIT(1'b0)) ff3 (.Q(DataOut[3]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[3]));
19
20 endmodule
```

O *testbench* (Listagem 15) usa um relógio de 100 MHz, como o da placa, e tem duas partes:

- **casos dirigidos**, que mostram uma a uma as propriedades do registo: valor inicial 0; com `CE = 0` não carrega; com `CE = 1` carrega no flanco ascendente e mantém o valor entre flancos; o `CLR` limpa o registo imediatamente, a meio de um ciclo, e tem prioridade sobre `CE`; depois de libertar o `CLR` o registo só volta a carregar no flanco seguinte; carregamento dos 16 valores possíveis e de um *bit* a 1 (e um *bit* a 0) em cada posição;
- **2000 ciclos com entradas aleatórias** (`Data`, `CE` e `CLR`, com o `CLR` a mudar em instantes que não coincidem com o relógio). A saída é comparada continuamente com um **modelo de referência** comportamental escrito no próprio *testbench*, e verifica-se que a saída nunca muda fora de um flanco ascendente, a não ser com `CLR` ativo.

Listagem 15: Testbench do registo de 4 bits (`registo4/registo4_teste1.v`)

```
1  `timescale 1ns / 1ps
2
3  // Testbench do registo de 4 bits com clear e enable.
4  // Parte 1: casos dirigidos (CE, clear assíncrono, prioridade do clear, 16 valores).
5  // Parte 2: 2000 ciclos com entradas aleatórias comparados com um modelo de referência,
6  //          com CLR a mudar em instantes que não coincidem com o relógio.
7  module registo4_teste1();
8
9  //   Declaração das entradas
10     reg [3:0] D = 4'h0;
11     reg CE = 0, CLR = 0;
```

```
12      reg Clk = 0;
13
14  //  Declaração da saída
15      wire [3:0] Q;
16
17  //  Instanciação da UUT e ligação das entradas e saídas
18      Registo4 UUT (
19          .Data(D),
20          .CE(CE),
21          .Clk(Clk),
22          .CLR(CLR),
23          .DataOut(Q)
24      );
25
26  //  Relógio de 100 MHz (período de 10ns), como o da Basys3
27      always #5 Clk = ~Clk;
28
29  //  Modelo de referência: o comportamento esperado do registo
30      reg [3:0] referencia = 4'h0;
31      always @(posedge Clk or posedge CLR)
32          if (CLR)      referencia <= 4'h0;
33          else if (CE)  referencia <= D;
34
35  //  Verificação contínua: a saída tem de ser sempre igual à referência
36  //  (comparada 1ns depois de qualquer mudança, para dar tempo à UUT)
37      integer erros = 0, verificacoes = 0;
38      always @(Q or referencia) begin
39          #1;
40          verificacoes = verificacoes + 1;
41          if (Q !== referencia) begin
42              if (erros < 10) $display("ERRO t=%0t: Q=%h (esperado %h) D=%h CE=%b CLR=%b", $time, Q,
    referencia, D, CE, CLR);
43              erros = erros + 1;
44          end
45      end
46
47  //  Verificação direta num instante (para os casos dirigidos)
48      task verifica(input [3:0] esperado, input [8*40:1] caso);
49          begin
50              verificacoes = verificacoes + 1;
51              if (Q !== esperado) begin
52                  $display("ERRO t=%0t (%0s): Q=%h (esperado %h)", $time, caso, Q, esperado);
53                  erros = erros + 1;
54              end
55          end
56      endtask
57
58  //  A saída só pode mudar num flanco ascendente do relógio ou com CLR ativo
59      time ultimo_flanco = -1;
60      always @(posedge Clk) ultimo_flanco = $time;
61      always @(Q)
62          if ($time > 0 && $time != ultimo_flanco && CLR !== 1'b1) begin
63              $display("ERRO t=%0t: Q mudou para %h fora do flanco do relogio e sem CLR", $time, Q);
64              erros = erros + 1;
65          end
66
67      integer n, v;
68      integer semente = 2026;        // semente fixa: resultado repetível
69      initial begin
70          // --- Parte 1: casos dirigidos ------------------------------------------
71          #1   verifica(4'h0, "valor inicial apos configuracao (INIT=0)");
72          #100;                                          // configurações iniciais da FPGA
```

```
73          // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
74          D = 4'h5; CE = 0;  #10 verifica(4'h0, "CE=0 nao carrega");
75          CE = 1;            #10 verifica(4'h5, "CE=1 carrega no flanco");
76          D = 4'hA;          #3  verifica(4'h5, "entre flancos a saida mantem-se");
77                             #7  verifica(4'hA, "carrega A");
78          CE = 0; D = 4'h3;  #50 verifica(4'hA, "CE=0 mantem durante 5 ciclos");
79          #2 CLR = 1;        #1  verifica(4'h0, "CLR assincrono (sem flanco)");
80          CE = 1; D = 4'hF;  #20 verifica(4'h0, "CLR tem prioridade sobre CE");
81          CLR = 0;           #0.5 verifica(4'h0, "apos CLR so carrega no flanco seguinte");
82                             #6.5 verifica(4'hF, "carrega F apos libertar CLR");
83          // Os 16 valores possíveis, carregados um a um (cada bit em 0 e em 1)
84          for (v = 0; v < 16; v = v + 1) begin
85              D = v; #10 verifica(v, "carregamento dos 16 valores");
86          end
87          // Bits independentes: 1 a andar e 0 a andar
88          for (v = 0; v < 4; v = v + 1) begin
89              D = 4'b0001 << v;     #10 verifica(4'b0001 << v, "um bit a 1");
90              D = ~(4'b0001 << v);  #10 verifica(~(4'b0001 << v), "um bit a 0");
91          end
92
93          // --- Parte 2: entradas aleatórias contra o modelo de referência ----------
94          for (n = 0; n < 2000; n = n + 1) begin
95              // instante aleatório entre 0 e 3ns depois do flanco descendente,
96              // nunca em cima do flanco ascendente (evita corridas no testbench)
97              @(negedge Clk); verifica(referencia, "ciclo aleatorio");
98              #({$random(semente)} % 4);
99              D   = $random(semente);
100             CE  = $random(semente);
101             CLR = ({$random(semente)} % 8 == 0);                  // clear em ~1/8 das vezes
102         end
103         #2 CLR = 0; #20;                                    // sem impulsos de largura nula no CLR
104
105         if (erros == 0) $display("Simulacao OK: Registo4 sem erros (%0d verificacoes)",
    verificacoes);
106         else            $display("Simulacao FALHOU: Registo4 com %0d erros", erros);
107         $finish;
108     end
109 endmodule
```

A Figura 15 mostra os casos dirigidos. Aos 101 ns `D` passa a 5 com `CE = 0` e a saída `Q` mantém-se a 0; quando `CE` passa a 1, `Q` carrega o 5 no flanco ascendente seguinte (115 ns) e o A no flanco de 125 ns. Com `CE = 0`, `Q` mantém o A durante cinco ciclos, embora `D` seja 3. Aos 183 ns, **entre dois flancos**, o `CLR` vai a 1 e `Q` passa imediatamente a 0 (*clear* assíncrono); durante o impulso de `CLR`, mesmo com `CE = 1` e `D = F`, o registo continua a 0. Só no flanco de 205 ns, depois de o `CLR` voltar a 0, carrega o F. Seguem-se os valores 0 a F e os padrões 1, E, 2, D, 4, B, 8, 7 (um *bit* a 1 e um *bit* a 0 em cada posição). O modelo de referência (`referencia`, na última linha) coincide sempre com `Q`. Na consola: `Simulacao OK: Registo4 sem erros (3056 verificacoes)`.

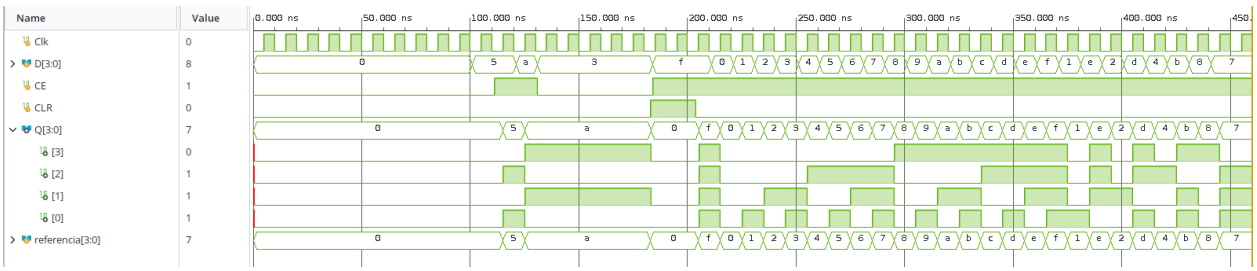


Figura 15: Simulação do registo de 4 bits no Vivado: casos dirigidos (0 a 460 ns).

7 Divisor de clock e contador de módulo 4

7.1 Divisor de clock

O relógio da Basys3 tem 100 MHz, uma frequência demasiado elevada para o varrimento dos *displays*. O divisor de *clock* `DivisorClock` (Listagem 16) conta ciclos de `ClockIn` e inverte `ClockOut` ao fim de `METADE` ciclos, pelo que `ClockOut` está `METADE` ciclos a 1 e `METADE` ciclos a 0 (*duty cycle* de 50 %):

$$f_out = f_in / (2 \times METADE) = 100\ MHz / (2 \times 25\,000) = 2\ kHz$$

O valor por omissão (`METADE = 25000`) dá 2 kHz, dentro do intervalo pedido (1 kHz a 10 kHz). `METADE` é um parâmetro do módulo, o que permite usar valores pequenos na simulação e, na placa, um valor muito maior para fazer *debug* visual (por exemplo, `METADE = 25_000_000` dá 2 Hz e permite ver cada *display* a acender individualmente). O módulo foi obtido simplificando os divisores `clk_div` e `Clock_Divider` dos exemplos Verilog do Moodle (secção 9.3): retiraram-se as entradas de *enable* e de *reset*, que não são necessárias neste projeto, e corrigiu-se a contagem para durar exatamente `METADE` ciclos.

Listagem 16: Divisor de clock (baseado nos exemplos do Moodle) (`divisorclock/divisorclock.v`)

```
1  `timescale 1ns / 1ps
2
3  // Divisor de clock: gera ClockOut a partir do relógio de 100 MHz da Basys3.
4  // ClockOut inverte a cada METADE ciclos de ClockIn, logo
5  // f_out = f_in / (2 * METADE) = 100 MHz / 50000 = 2 kHz (entre 1 e 10 kHz).
6  // Para debug visual (displays a piscar) usar, por exemplo, METADE = 25_000_000 (2 Hz).
7  // Baseado nos divisores clk_div e Clock_Divider dos exemplos Verilog do Moodle,
8  // sem enable e reset e com a contagem corrigida (relatório, secção 9.3).
9  module DivisorClock #(
10     parameter METADE = 25000
11     )(
12     input ClockIn,
13     output reg ClockOut = 1'b0
14     );
15
16      reg [31:0] contagem = 0;
17
18      always @(posedge ClockIn)
19          if (contagem == METADE - 1) begin
20              contagem <= 0;
21              ClockOut <= ~ClockOut;
22          end
23          else
24              contagem <= contagem + 1;
25
26 endmodule
```

O *testbench* (Listagem 17) instancia o mesmo divisor com `METADE` igual a 1, 2, 5 e 7 e, em todos os períodos (pelo menos 50 por instância), mede o tempo a 1 e o tempo a 0 da saída e confirma que são exatamente `METADE` ciclos de 10 ns e que a saída só muda nos flancos ascendentes de `ClockIn`. Simula também uma instância com o valor da placa (25 000) durante 12 períodos de saída (6 ms de simulação) e confirma que a saída está 250 000 ns a 1 e 250 000 ns a 0, ou seja, que a frequência é 2000 Hz, dentro do intervalo pedido.

Listagem 17: Testbench do divisor de clock (`divisorclock/divisorclock_teste1.v`)

```
1  `timescale 1ns / 1ps
2
3  // Testbench do divisor de clock.
4  // Verifica, para vários valores do parâmetro METADE, que ClockOut:
5  //   - tem período 2 x METADE ciclos de ClockIn (f_out = f_in / (2 x METADE));
```

```
//   - tem duty cycle de 50% (METADE ciclos a 1 e METADE ciclos a 0);
//   - só muda no flanco ascendente de ClockIn;
//   - mantém esse comportamento ao longo de muitos períodos.
// Inclui o valor usado na placa (METADE = 25000), simulando 12 períodos de saída (6 ms),
// e confirma que a frequência real fica entre 1 kHz e 10 kHz.
module divisorclock_teste1();

//   Declaração das entradas: relógio de 100 MHz (período de 10ns), como o da Basys3
     reg Clk = 0;
     always #5 Clk = ~Clk;

//   Declaração das saídas
     wire Out1, Out2, Out5, Out7, OutPlaca;

//   Instanciação das UUT com vários valores de METADE (o circuito é o mesmo)
     DivisorClock #(.METADE(1))  UUT1 (.ClockIn(Clk), .ClockOut(Out1));
     DivisorClock #(.METADE(2))  UUT2 (.ClockIn(Clk), .ClockOut(Out2));
     DivisorClock #(.METADE(5))  UUT5 (.ClockIn(Clk), .ClockOut(Out5));
     DivisorClock #(.METADE(7))  UUT7 (.ClockIn(Clk), .ClockOut(Out7));
     DivisorClock                UUTP (.ClockIn(Clk), .ClockOut(OutPlaca));   // valor por omissão

//   Instante do último flanco ascendente de ClockIn
     time flanco_in = 0;
     always @(posedge Clk) flanco_in = $time;

     integer erros = 0;

//   Monitor genérico de uma saída: mede tempo a 1, tempo a 0 e período
     task automatic monitoriza(input integer metade, input integer periodos_min, input [8*12:1]
nome);
         time t_sobe, t_desce, t_sobe_ant, alto, baixo;
         integer p;
         begin
             p = 0;
             t_sobe_ant = 0;
             while (p < periodos_min + 1) begin
                 case (metade)
                     1: @(posedge Out1);  2: @(posedge Out2);  5: @(posedge Out5);
                     7: @(posedge Out7);  default: @(posedge OutPlaca);
                 endcase
                 t_sobe = $time;
                 if (t_sobe != flanco_in) begin
                     $display("ERRO %0s: ClockOut subiu fora do flanco de ClockIn (t=%0t)", nome,
t_sobe);
                     erros = erros + 1;
                 end
                 if (p > 0) begin
                     alto  = t_desce - t_sobe_ant;
                     baixo = t_sobe - t_desce;
                     if (alto != metade*10 || baixo != metade*10) begin
                         $display("ERRO %0s: a 1 durante %0t ns e a 0 durante %0t ns (esperado %0d
ns cada)",
                                  nome, alto, baixo, metade*10);
                         erros = erros + 1;
                     end
                 end
                 case (metade)
                     1: @(negedge Out1);  2: @(negedge Out2);  5: @(negedge Out5);
                     7: @(negedge Out7);  default: @(negedge OutPlaca);
                 endcase
                 t_desce = $time;
                 if (t_desce != flanco_in) begin
```

```
                    $display("ERRO %0s: ClockOut desceu fora do flanco de ClockIn (t=%0t)", nome, t_desce);
                    erros = erros + 1;
                end
                t_sobe_ant = t_sobe;
                p = p + 1;
            end
        end
    endtask

    reg fim1 = 0, fim2 = 0, fim5 = 0, fim7 = 0, fimP = 0;
    initial begin monitoriza(1, 50, "METADE=1");  fim1 = 1; end
    initial begin monitoriza(2, 50, "METADE=2");  fim2 = 1; end
    initial begin monitoriza(5, 50, "METADE=5");  fim5 = 1; end
    initial begin monitoriza(7, 50, "METADE=7");  fim7 = 1; end
    initial begin monitoriza(25000, 12, "placa");  fimP = 1; end

    real freq;
    initial begin
        wait (fim1 && fim2 && fim5 && fim7 && fimP);
        freq = 100.0e6 / (2 * UUTP.METADE);
        $display("Frequencia de ClockOut na placa: %0.1f Hz", freq);
        if (freq < 1000.0 || freq > 10000.0) begin
            $display("ERRO: frequencia fora do intervalo 1 kHz a 10 kHz");
            erros = erros + 1;
        end
        if (erros == 0) $display("Simulacao OK: DivisorClock sem erros");
        else            $display("Simulacao FALHOU: DivisorClock com %0d erros", erros);
        $finish;
    end

//   Segurança: termina se alguma saída nunca chegar a mudar
    initial begin
        #10_000_000;
        $display("Simulacao FALHOU: DivisorClock nao produziu os periodos esperados (fim1=%b fim2=%b fim5=%b fim7=%b placa=%b)",
                 fim1, fim2, fim5, fim7, fimP);
        $finish;
    end
endmodule
```

Na Figura 16 vê-se que `Out1` muda a cada ciclo de `Clk` (período de 20 ns), `Out2` a cada 2 ciclos, `Out5` a cada 5 e `Out7` a cada 7 ciclos (período de 140 ns), sempre num flanco ascendente de `Clk`. O contador interno da instância com `METADE = 7` conta de 0 a 6 e volta a 0, momento em que a saída é invertida. Resultado: `Simulacao OK: DivisorClock sem erros`.

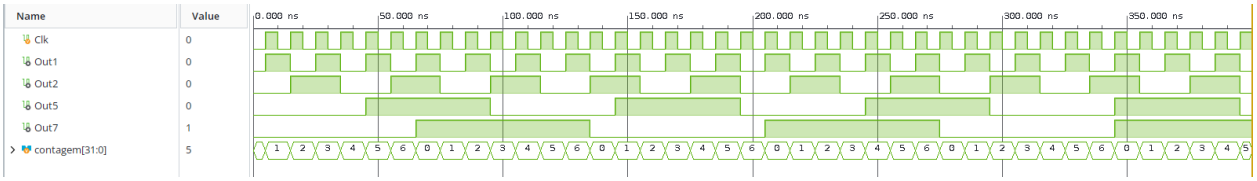


Figura 16: Simulação do divisor de clock no Vivado para METADE = 1, 2, 5 e 7 (0 a 400 ns).

7.2 Contador de módulo 4

O contador `Contador4` (Listagem 18) é um registo de 2 *bits* que soma 1 em cada flanco ascendente de `Clock`; como 3 + 1 = 4 não cabe em 2 *bits*, o estado volta naturalmente a 0, o que dá a sequência 0, 1, 2, 3, 0, … O módulo foi obtido a partir do "contador simples" dos exemplos Verilog do Moodle, retirando o que não era necessário (secção 9.4). As saídas `Q1` e `Q0` são as variáveis de varrimento `Control1` e

`Control0` da Fig. 31, que atuam na seleção dos *multiplexers* e nas entradas do descodificador. Com o relógio de 2 kHz do divisor, cada *display* fica ativo durante 0,5 ms e o ciclo completo dos quatro *displays* repete-se 500 vezes por segundo; pela persistência da imagem na retina, o observador tem a sensação de que os quatro *displays* estão sempre acesos.

Listagem 18: Contador de módulo 4 (baseado no exemplo do Moodle) (`contador4/contador4.v`)

```
`timescale 1ns / 1ps

// Contador de módulo 4 (0, 1, 2, 3, 0, ...) que avança no flanco ascendente de Clock.
// Q1 Q0 vão atuar nas seleções dos multiplexers e nas entradas do descodificador.
// Baseado no "Contador simples" dos exemplos Verilog do Moodle,
// reduzido a 2 bits com atribuição não bloqueante (relatório, secção 9.4).
module Contador4(
    input Clock,
    output Q1,
    output Q0
    );

     reg [1:0] estado = 2'b00;

     always @(posedge Clock)
         estado <= estado + 1;     // 3 + 1 = 0 em 2 bits: módulo 4

     assign Q1 = estado[1];
     assign Q0 = estado[0];

endmodule
```

O *testbench* (Listagem 19) verifica o estado inicial (0) e a sequência durante 100 flancos (25 voltas completas); confirma também que o estado não muda no flanco descendente e que as saídas mudam exatamente uma vez por ciclo, sempre no instante de um flanco ascendente. Na Figura 17 observa-se a sequência 0, 1, 2, 3, 0, … no estado interno, com `Q0` a mudar em todos os flancos ascendentes e `Q1` a cada dois. Resultado: `Simulacao OK: Contador4 sem erros (100 ciclos)`.

Listagem 19: Testbench do contador de módulo 4 (`contador4/contador4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do contador de módulo 4.
// Verifica:
//   - o estado inicial (0 após a configuração da FPGA);
//   - a sequência 0, 1, 2, 3, 0, ... durante 100 flancos (25 voltas);
//   - que as saídas só mudam no flanco ascendente (nunca no descendente nem entre flancos);
//   - que muda exatamente uma vez por ciclo de relógio.
module contador4_teste1();

//   Declaração das entradas
     reg Clk = 0;

//   Declaração das saídas
     wire Q1, Q0;

//   Instanciação da UUT e ligação das entradas e saídas
     Contador4 UUT (
         .Clock(Clk),
         .Q1(Q1),
         .Q0(Q0)
     );

//   Relógio com período de 20ns
```

```
25      always #10 Clk = ~Clk;
26
27  //  As saídas só podem mudar no instante de um flanco ascendente
28      time flanco = -1;
29      integer erros = 0, mudancas = 0;
30      always @(posedge Clk) flanco = $time;
31      always @({Q1, Q0})
32          if ($time > 0) begin
33              mudancas = mudancas + 1;
34              if ($time != flanco) begin
35                  $display("ERRO t=%0t: Q1Q0 mudou para %b%b fora do flanco ascendente", $time, Q1,
    Q0);
36                  erros = erros + 1;
37              end
38          end
39
40  //  Verificação do estado depois de cada flanco ascendente
41      reg [1:0] esperado = 2'b00;
42      integer n;
43      initial begin
44          #1;
45          if ({Q1, Q0} !== 2'b00) begin
46              $display("ERRO: estado inicial %b%b (esperado 00)", Q1, Q0);
47              erros = erros + 1;
48          end
49          for (n=0; n<100; n=n+1) begin
50              @(posedge Clk); #1;
51              esperado = esperado + 1;           // módulo 4: 3 + 1 = 0 em 2 bits
52              if ({Q1, Q0} !== esperado) begin
53                  $display("ERRO t=%0t: Q1Q0=%b%b (esperado %b)", $time, Q1, Q0, esperado);
54                  erros = erros + 1;
55              end
56              @(negedge Clk); #1;                // a meio do ciclo continua igual
57              if ({Q1, Q0} !== esperado) begin
58                  $display("ERRO t=%0t: Q1Q0 mudou no flanco descendente", $time);
59                  erros = erros + 1;
60              end
61          end
62          // Em 100 ciclos: Q0 muda 100 vezes e Q1 50 vezes; o bloco always conta
63          // eventos de mudança de {Q1,Q0}, que são 100 (um por ciclo)
64          if (mudancas != 100) begin
65              $display("ERRO: %0d mudancas em 100 ciclos (esperado 100)", mudancas);
66              erros = erros + 1;
67          end
68          if (erros == 0) $display("Simulacao OK: Contador4 sem erros (100 ciclos)");
69          else            $display("Simulacao FALHOU: Contador4 com %0d erros", erros);
70          $finish;
71      end
72  endmodule
```

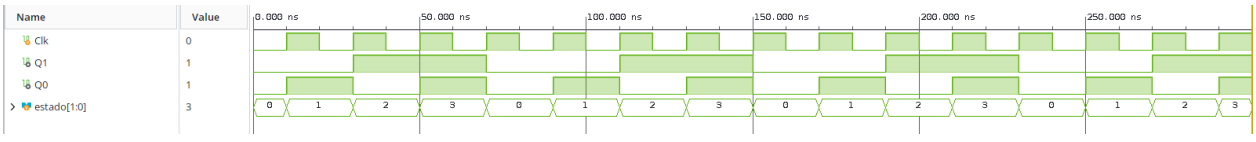


Figura 17: Simulação do contador de módulo 4 no Vivado (0 a 300 ns).

8 Controlador dos displays da placa Basys3

O controlador `ControloDisplays` (Listagem 20) integra, conforme a Fig. 31 do enunciado, os módulos desenvolvidos, de modo a apresentar, de forma consecutiva, um algarismo ou símbolo independente em cada um dos quatro *displays*. Os módulos das aulas anteriores foram reutilizados **sem alterações**, bastando adicionar os respetivos ficheiros ao novo projeto Vivado `controlodisplays`. A Tabela 8 resume a função de cada instância.

Tabela 8: Módulos que constituem o controlador dos displays.

Instância	Módulo	Função
reg0 a reg3	Registo4	Guardam o código do símbolo de cada *display* (carga com Load*i*, *clear* com Clr*i*)
regDP	Registo4	Guarda o estado dos quatro pontos decimais (1 = ponto aceso)
divisor	DivisorClock	Reduz a frequência de 100 MHz para 2 kHz (relógio do varrimento)
contador	Contador4	Gera as variáveis de varrimento Control1 e Control0 (0, 1, 2, 3, 0, …)
mux16x4	multiplexer16x4	Liga ao transcodificador o registo do *display* ativo
transcod	Transcod7Seg	Converte o símbolo nos segmentos a acender (partilhados por todos os *displays*)
mux4x1	multiplexer4x1	Escolhe o ponto decimal do *display* ativo
descod	descodificador2x4	Ativa o ânodo do *display* selecionado, se o respetivo Enable estiver a 1

Na integração houve três aspetos a que foi preciso dar atenção:

1. **Polaridade dos ânodos.** Na Fig. 31 as saídas do descodificador estão ligadas diretamente aos ânodos. No entanto, como alerta o enunciado, os *displays* da Basys3 são de ânodo comum e, segundo o manual da placa, cada ânodo é alimentado através de um transístor PNP, pelo que os sinais AN0 a AN3 são **ativos a Low**. Como o descodificador tem saídas ativas a *High*, ligá-lo diretamente faria acender os três *displays* não selecionados (todos com o símbolo do *display* selecionado) e apagaria o selecionado. Por isso as saídas do descodificador são negadas antes de chegarem aos ânodos (`assign Anodos = ~DisplayAtivo`). Com esta negação, um *enable* a 0 apaga efetivamente o respetivo *display*.
2. **Polaridade do ponto decimal.** O registo dos pontos guarda 1 para "ponto aceso", que é a convenção mais natural para o utilizador; como o segmento DP também é ativo a *Low*, a saída do *multiplexer* 4x1 é negada, exatamente como na Fig. 31. Os segmentos não precisam de negação porque o transcodificador já tem saídas ativas a *Low*.
3. **Nomes das portas.** Na Fig. 31 os *multiplexers* aparecem com portas `In0`…`In3` e `OutMux`; como se reutilizaram os módulos das secções 2 a 4, as ligações foram feitas aos nomes reais desses módulos (`DataIn0`…`DataIn3`, `DataOut`, `Sel`, `Bit1`, `Bit0`, `En0`…`En3`, `Out0`…`Out3`), sem alterar a função. A ordem dos registos respeita a figura: o registo *i* está ligado à entrada *i* do *multiplexer* 16x4, pelo que o *display* controlado por `Anodos[i]` mostra sempre o registo *i* (`Anodos[0]` é o *display* da direita).

Os cinco registos usam o relógio de 100 MHz da placa e carregam apenas quando o respetivo sinal `Load` está a 1; o contador usa o relógio dividido. O parâmetro `METADE` do controlador é passado ao divisor, o que permite reduzi-lo nos *testbenches*.

Listagem 20: Controlador dos quatro displays (Fig. 31) (`controlodisplays/controlodisplays.v`)

```
1  `timescale 1ns / 1ps
2
3  // Controlador dos 4 displays de 7 segmentos da Basys3 (Fig. 31 do enunciado).
4  // Reutiliza os módulos já desenvolvidos:
5  //   - 5 x Registo4: 4 registos com o símbolo de cada display e 1 com os pontos decimais;
6  //   - multiplexer16x4: liga ao transcodificador o registo do display ativo;
7  //   - multiplexer4x1: escolhe o ponto decimal do display ativo;
```

```
//   - Transcod7Seg: converte o símbolo nos segmentos a acender;
//   - descodificador2x4: escolhe o display ativo (com um enable por display);
//   - DivisorClock + Contador4: geram as variáveis de varrimento Control1 e Control0.
// O display i (Anodos[i]) mostra o registo i; Anodos[0] é o display da direita.
// Segmentos, ponto decimal e ânodos são ativos a Low na Basys3.
module ControloDisplays #(
    parameter METADE = 25000        // varrimento a 100 MHz / (2 x 25000) = 2 kHz
    )(
    input [3:0] Data3,
    input Load3,
    input Clr3,
    input Enable3,
    input [3:0] Data2,
    input Load2,
    input Clr2,
    input Enable2,
    input [3:0] Data1,
    input Load1,
    input Clr1,
    input Enable1,
    input [3:0] Data0,
    input Load0,
    input Clr0,
    input Enable0,
    input [3:0] DotPoints,
    input LoadDP,
    input ClrDP,
    input Clock,
    output [6:0] Segmentos,
    output DotPoint,
    output [3:0] Anodos
    );

//   Sinais internos
     wire [3:0] Valor3, Valor2, Valor1, Valor0;   // conteúdo dos registos dos símbolos
     wire [3:0] Pontos;                           // conteúdo do registo dos pontos decimais
     wire [3:0] ValorAtivo;                       // símbolo do display ativo
     wire PontoAtivo;                             // ponto decimal do display ativo (1 = aceso)
     wire [3:0] DisplayAtivo;                     // saídas do descodificador (ativas a High)
     wire ClockVarrimento;                        // relógio dividido
     wire Control1, Control0;                     // variáveis de varrimento

//   Registos dos símbolos e dos pontos decimais (relógio de 100 MHz, carga com Load)
     Registo4 reg3  (.Data(Data3),     .CE(Load3),  .Clk(Clock), .CLR(Clr3),  .DataOut(Valor3));
     Registo4 reg2  (.Data(Data2),     .CE(Load2),  .Clk(Clock), .CLR(Clr2),  .DataOut(Valor2));
     Registo4 reg1  (.Data(Data1),     .CE(Load1),  .Clk(Clock), .CLR(Clr1),  .DataOut(Valor1));
     Registo4 reg0  (.Data(Data0),     .CE(Load0),  .Clk(Clock), .CLR(Clr0),  .DataOut(Valor0));
     Registo4 regDP (.Data(DotPoints), .CE(LoadDP), .Clk(Clock), .CLR(ClrDP), .DataOut(Pontos));

//   Varrimento: divisor de clock e contador de módulo 4
     DivisorClock #(.METADE(METADE)) divisor (
         .ClockIn(Clock),
         .ClockOut(ClockVarrimento)
     );

     Contador4 contador (
         .Clock(ClockVarrimento),
         .Q1(Control1),
         .Q0(Control0)
     );

//   Multiplexer 16x4: registo do display ativo para o transcodificador
```

```
70      multiplexer16x4 mux16x4 (
71          .DataIn0(Valor0),
72          .DataIn1(Valor1),
73          .DataIn2(Valor2),
74          .DataIn3(Valor3),
75          .DataOut(ValorAtivo),
76          .Sel({Control1, Control0})
77      );
78
79  //   Transcodificador: segmentos partilhados por todos os displays (ativos a Low)
80      Transcod7Seg transcod (
81          .Numero(ValorAtivo),
82          .Segmentos(Segmentos)
83      );
84
85  //   Multiplexer 4x1: ponto decimal do display ativo
86      multiplexer4x1 mux4x1 (
87          .Sel1(Control1),
88          .Sel0(Control0),
89          .DataIn0(Pontos[0]),
90          .DataIn1(Pontos[1]),
91          .DataIn2(Pontos[2]),
92          .DataIn3(Pontos[3]),
93          .DataOut(PontoAtivo)
94      );
95
96      assign DotPoint = ~PontoAtivo;     // o segmento DP acende com nível Low
97
98  //   Descodificador 2x4: ativa um display de cada vez, se o respetivo enable estiver a 1
99      descodificador2x4 descod (
100         .Bit1(Control1),
101         .Bit0(Control0),
102         .En0(Enable0),
103         .En1(Enable1),
104         .En2(Enable2),
105         .En3(Enable3),
106         .Out0(DisplayAtivo[0]),
107         .Out1(DisplayAtivo[1]),
108         .Out2(DisplayAtivo[2]),
109         .Out3(DisplayAtivo[3])
110     );
111
112     assign Anodos = ~DisplayAtivo;     // ânodos comuns ligados por transistores PNP: ativos a Low
113
114 endmodule
```

O *testbench* do controlador (Listagem 21) usa `METADE = 5`, pelo que cada *display* fica ativo durante 2 × 5 ciclos de 10 ns = 100 ns e o varrimento completo demora 400 ns. Inclui um modelo de referência dos cinco registos e, a meio de cada ciclo de relógio, compara as três saídas com o que é esperado para a seleção atual: ânodo do *display* selecionado a 0 (se o *enable* estiver a 1) e os outros a 1, segmentos do símbolo guardado nesse registo e ponto decimal desse *display*. Verifica ainda que a seleção avança uma posição exatamente a cada 10 ciclos. A sequência aplicada é a seguinte:

- registos a 0 e *displays* desligados; depois liga os quatro *enables* ("0000");
- carrega os quatro registos ao mesmo tempo (registos 3 a 0) com 2, 0, 2 e 6 ("2026") e o registo dos pontos com 0100 ("20.26");
- carrega só o registo 0 com 7 ("20.27") e muda `Data1` sem `Load1` (o *display* 1 não muda);

- carrega cada registo **sozinho**, com valores diferentes nas outras entradas de dados, até obter "rd.LE";
- desliga o *enable* do *display* 2, faz um *clear* assíncrono do registo 3 a meio de um ciclo, apaga os pontos, limpa todos os registos e, por fim, desliga todos os *displays*.

Listagem 21: Testbench do controlador dos displays (`controlodisplays/controlodisplays_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do controlador dos displays (ControloDisplays).
// Usa METADE = 5 para o varrimento ser visível em poucas centenas de ns:
// cada display fica ativo durante 2 x 5 ciclos de 10ns = 100ns.
// Verifica continuamente (a meio de cada ciclo de relógio) que:
//   - só o ânodo do display selecionado está ativo (a Low), e só se o enable estiver a 1;
//   - os segmentos são os do símbolo guardado no registo desse display;
//   - o ponto decimal é o bit desse display no registo dos pontos;
//   - o varrimento avança 0, 1, 2, 3, 0, ... de 100 em 100ns.
module controlodisplays_teste1();

//   Declaração das entradas
     reg [3:0] D3 = 0, D2 = 0, D1 = 0, D0 = 0, DP = 0;
     reg [3:0] Load = 0, Clr = 0, En = 0;      // índice i = display i
     reg LoadDP = 0, ClrDP = 0;
     reg Clk = 0;

//   Declaração das saídas
     wire [6:0] Segmentos;
     wire DotPoint;
     wire [3:0] Anodos;

//   Instanciação da UUT e ligação das entradas e saídas
     ControloDisplays #(.METADE(5)) UUT (
         .Data3(D3), .Load3(Load[3]), .Clr3(Clr[3]), .Enable3(En[3]),
         .Data2(D2), .Load2(Load[2]), .Clr2(Clr[2]), .Enable2(En[2]),
         .Data1(D1), .Load1(Load[1]), .Clr1(Clr[1]), .Enable1(En[1]),
         .Data0(D0), .Load0(Load[0]), .Clr0(Clr[0]), .Enable0(En[0]),
         .DotPoints(DP), .LoadDP(LoadDP), .ClrDP(ClrDP),
         .Clock(Clk),
         .Segmentos(Segmentos),
         .DotPoint(DotPoint),
         .Anodos(Anodos)
     );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
     always #5 Clk = ~Clk;

//   Seleção atual do varrimento (variáveis Control1 e Control0 do controlador)
     wire [1:0] Sel = {UUT.Control1, UUT.Control0};

//   Segmentos acesos (ativo a High) de cada símbolo, a partir da Fig. 30 e de "rdLEC-"
     reg [6:0] acesos [0:15];
     initial begin           //  abcdefg
         acesos[0]  = 7'b1111110;  acesos[1]  = 7'b0110000;
         acesos[2]  = 7'b1101101;  acesos[3]  = 7'b1111001;
         acesos[4]  = 7'b0110011;  acesos[5]  = 7'b1011011;
         acesos[6]  = 7'b1011111;  acesos[7]  = 7'b1110000;
         acesos[8]  = 7'b1111111;  acesos[9]  = 7'b1111011;
         acesos[10] = 7'b0000101;  acesos[11] = 7'b0111101;
         acesos[12] = 7'b0001110;  acesos[13] = 7'b1001111;
         acesos[14] = 7'b1001110;  acesos[15] = 7'b0000001;
     end

```

```
56 //   Modelo de referência dos 5 registos (clear assíncrono, carga no flanco ascendente)
57      reg [3:0] ref3 = 0, ref2 = 0, ref1 = 0, ref0 = 0, refDP = 0;
58      always @(posedge Clk or posedge Clr[3]) if (Clr[3]) ref3 <= 0; else if (Load[3]) ref3 <= D3;
59      always @(posedge Clk or posedge Clr[2]) if (Clr[2]) ref2 <= 0; else if (Load[2]) ref2 <= D2;
60      always @(posedge Clk or posedge Clr[1]) if (Clr[1]) ref1 <= 0; else if (Load[1]) ref1 <= D1;
61      always @(posedge Clk or posedge Clr[0]) if (Clr[0]) ref0 <= 0; else if (Load[0]) ref0 <= D0;
62      always @(posedge Clk or posedge ClrDP)  if (ClrDP)  refDP <= 0; else if (LoadDP) refDP <= DP;
63
64 //   Verificação contínua, a meio de cada ciclo de relógio:
65 //   compara as saídas com as esperadas para a seleção atual
66      reg [3:0] valor;
67      reg [6:0] seg_esperado;
68      reg       dp_esperado;
69      reg [3:0] an_esperado;
70      integer erros = 0, verificacoes = 0;
71      always @(negedge Clk) if ($time > 0) begin   // em t=0 a inicialização de Clk conta como
    flanco
72          case (Sel)
73              2'd0: valor = ref0;
74              2'd1: valor = ref1;
75              2'd2: valor = ref2;
76              2'd3: valor = ref3;
77          endcase
78          seg_esperado = ~acesos[valor];
79          dp_esperado  = ~refDP[Sel];
80          an_esperado  = ~((4'b0001 << Sel) & En);
81          verificacoes = verificacoes + 1;
82          if (Segmentos !== seg_esperado || DotPoint !== dp_esperado || Anodos !== an_esperado)
    begin
83              if (erros < 10)
84                  $display("ERRO t=%0t: Sel=%0d Seg=%b DP=%b An=%b (esperado Seg=%b DP=%b An=%b)",
85                           $time, Sel, Segmentos, DotPoint, Anodos, seg_esperado, dp_esperado,
    an_esperado);
86              erros = erros + 1;
87          end
88      end
89
90 //   Verificação do varrimento: a seleção avança 1 (módulo 4) a cada 2 x METADE ciclos
91      integer ciclos = 0, trocas = 0;
92      reg [1:0] sel_ant = 0;
93      always @(posedge Clk) begin
94          #1;
95          ciclos = ciclos + 1;
96          if (Sel !== sel_ant) begin
97              if (Sel !== sel_ant + 2'd1) begin
98                  $display("ERRO t=%0t: varrimento passou de %0d para %0d", $time, sel_ant, Sel);
99                  erros = erros + 1;
100             end
101             if (trocas > 0 && ciclos != 10) begin
102                 $display("ERRO t=%0t: display ativo durante %0d ciclos (esperado 10)", $time,
    ciclos);
103                 erros = erros + 1;
104             end
105             trocas = trocas + 1;
106             ciclos = 0;
107             sel_ant = Sel;
108         end
109     end
110
111 //   Impulso de carga com a duração de um ciclo de relógio (como uma tecla sincronizada)
112     task carrega(input [3:0] quais, input [3:0] v3, v2, v1, v0);
113         begin
```

```
114             D3 = v3; D2 = v2; D1 = v1; D0 = v0;
115             Load = quais;
116             #10 Load = 4'b0000;
117         end
118     endtask
119
120 //   Definição do valor das entradas ao longo do tempo
121 //   (as entradas mudam sempre 2ns depois de um flanco descendente)
122     initial begin
123         #202;                                   // configurações iniciais da FPGA
124                                                 // registos a 0 e displays desligados
125         En = 4'b1111;             #400;        // 4 displays ligados: "0000"
126         carrega(4'b1111, 4'd2, 4'd0, 4'd2, 4'd6);
127                                   #400;        // "2026"
128         DP = 4'b0100; LoadDP = 1;  #10 LoadDP = 0;
129                                   #400;        // "20.26" (ponto no display 2)
130         carrega(4'b0001, 4'd0, 4'd0, 4'd0, 4'd7);
131                                   #400;        // "20.27": só o display 0 muda
132         D1 = 4'd9;                #400;        // sem Load o registo não muda
133         // cada registo carregado sozinho, com valores diferentes nas outras entradas
134         carrega(4'b1000, 4'd10, 4'd11, 4'd12, 4'd13);   // só o display 3: "r0.27"
135         carrega(4'b0100, 4'd10, 4'd11, 4'd12, 4'd13);   // só o display 2: "rd.27"
136         carrega(4'b0010, 4'd10, 4'd11, 4'd12, 4'd13);   // só o display 1: "rd.L7"
137         carrega(4'b0001, 4'd10, 4'd11, 4'd12, 4'd13);   // só o display 0: "rd.LE"
138                                   #400;        // "rd.LE" (símbolos originais)
139         En = 4'b1011;             #400;        // display 2 desligado: "r LE"
140         #3 Clr[3] = 1; #4 Clr[3] = 0;          // clear assíncrono, entre flancos
141                                   #400;        // "0 LE"
142         En = 4'b1111; ClrDP = 1;   #10 ClrDP = 0;
143                                   #400;        // pontos apagados: "0dLE"
144         Clr = 4'b1111;            #10 Clr = 4'b0000;
145                                   #400;        // "0000"
146         En = 4'b0000;             #400;        // todos os displays desligados
147
148         if (trocas < 40) begin
149             $display("ERRO: so houve %0d mudancas de display", trocas);
150             erros = erros + 1;
151         end
152         if (erros == 0) $display("Simulacao OK: ControloDisplays sem erros (%0d verificacoes)",
    verificacoes);
153         else            $display("Simulacao FALHOU: ControloDisplays com %0d erros", erros);
154         $finish;
155     end
156 endmodule
```

A Figura 18 mostra a simulação completa: a seleção `Sel` (Control1 Control0) percorre continuamente 0, 1, 2, 3, ao ritmo do relógio de varrimento, e `ValorAtivo`, à saída do *multiplexer* 16x4, mostra em cada instante o registo do *display* ativo. A Figura 19 amplia o intervalo em que se mostra "20.26" e depois "20.27": `Anodos` tem sempre um único *bit* a 0, que acompanha `Sel` (1110 para o *display* 0, …, 0111 para o *display* 3); os segmentos correspondem ao símbolo desse *display* (por exemplo, 0010010 para o algarismo 2 e 0100000 para o 6) e `DotPoint` só vai a 0 quando está ativo o *display* 2. A Figura 20 mostra o mesmo varrimento depois de cada registo ter sido carregado sozinho com os símbolos originais: os *displays* 0 a 3 apresentam E, L, d e r (segmentos 0110000, 1110001, 1000010 e 1111010), ou seja, "rd.LE" lido da esquerda para a direita, com o ponto no *display* 2. Resultado: `Simulacao OK: ControloDisplays sem erros (469 verificacoes)`.

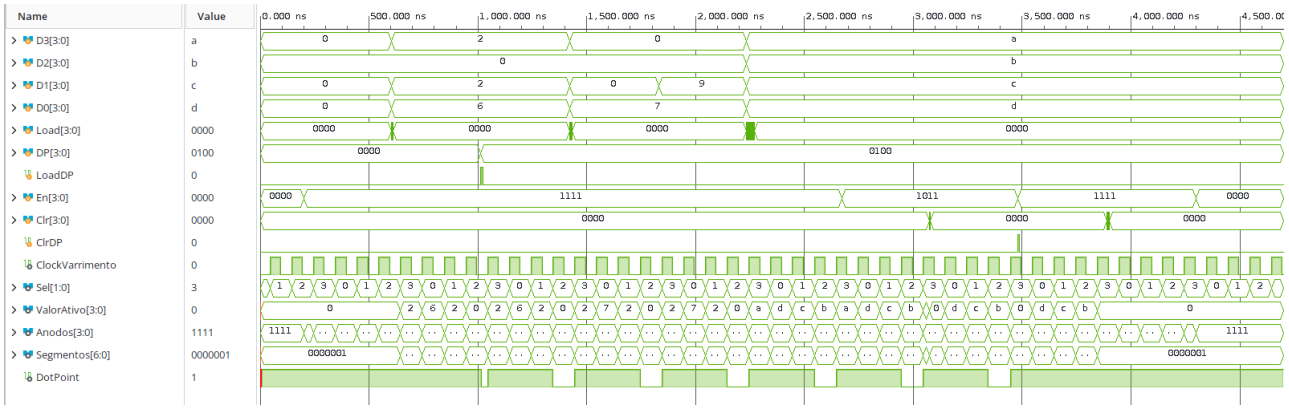


Figura 18: Simulação completa do controlador dos displays no Vivado (METADE = 5).

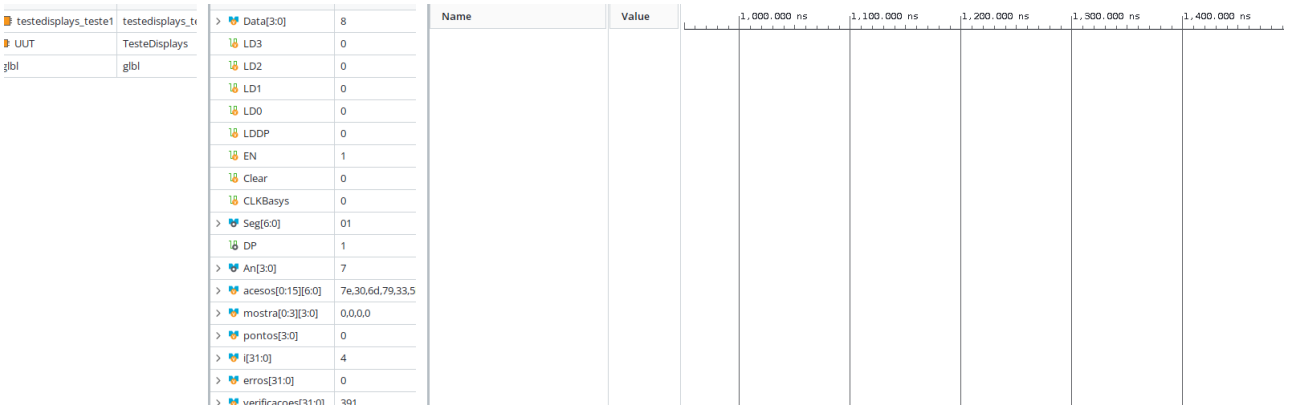


Figura 19: Pormenor da simulação do controlador: varrimento dos quatro displays com "20.26" e "20.27".

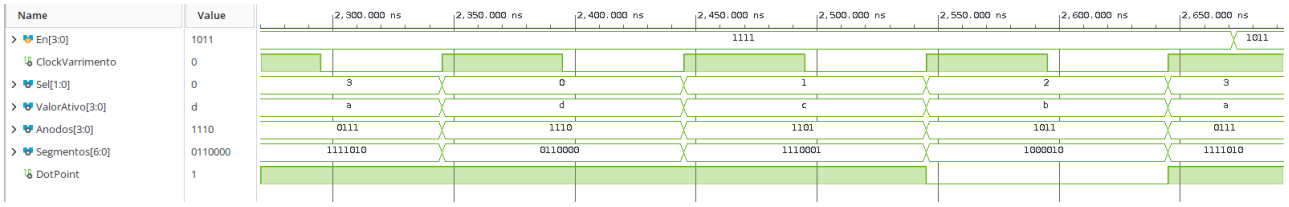


Figura 20: Pormenor da simulação do controlador: varrimento dos quatro displays com "rd.LE".

8.1 Circuito de teste na placa

Para o ensaio na placa criou-se o módulo `TesteDisplays` (Fig. 32 do enunciado e Listagem 22): o mesmo barramento `Data` de 4 *bits* alimenta os quatro registos dos símbolos e o registo dos pontos decimais, e os sinais `LD3` a `LD0` e `LDDP` escolhem qual dos registos é carregado. `Clear` limpa todos os registos e `EN` liga ou desliga os quatro *displays* em simultâneo. O relógio `CLKBasys` é o oscilador de 100 MHz da placa.

Listagem 22: Circuito de teste do controlador na Basys3 (Fig. 32) (`controlodisplays/testedisplays.v`)

```
1  `timescale 1ns / 1ps
2
3  // Circuito de teste do controlador dos displays na Basys3 (Fig. 32 do enunciado).
4  // O mesmo barramento Data alimenta os 4 registos dos símbolos e o registo dos pontos
5  // decimais; cada registo só carrega quando o respetivo sinal LDx está a 1.
6  // Clear limpa todos os registos e EN liga ou desliga os 4 displays.
7  module TesteDisplays #(
8      parameter METADE = 25000        // passado ao divisor de clock do controlador
9      )(
10     input [3:0] Data,
11     input LD3,
12     input LD2,
13     input LD1,
```

```
14      input LD0,
15      input LDDP,
16      input EN,
17      input Clear,
18      input CLKBasys,
19      output [6:0] Seg,
20      output DP,
21      output [3:0] An
22      );
23
24       ControloDisplays #(.METADE(METADE)) controlo (
25           .Data3(Data),  .Load3(LD3),  .Clr3(Clear),  .Enable3(EN),
26           .Data2(Data),  .Load2(LD2),  .Clr2(Clear),  .Enable2(EN),
27           .Data1(Data),  .Load1(LD1),  .Clr1(Clear),  .Enable1(EN),
28           .Data0(Data),  .Load0(LD0),  .Clr0(Clear),  .Enable0(EN),
29           .DotPoints(Data),
30           .LoadDP(LDDP),
31           .ClrDP(Clear),
32           .Clock(CLKBasys),
33           .Segmentos(Seg),
34           .DotPoint(DP),
35           .Anodos(An)
36       );
37
38  endmodule
```

Na escolha dos pinos (Tabela 9 e Listagem 23) procurou-se que o ensaio fosse intuitivo: o valor a carregar fica nos interruptores SW3 a SW0, e os interruptores de carga SW15 a SW12 estão pela mesma ordem dos *displays* (SW15 carrega o *display* da esquerda e SW12 o da direita). Como os registos são carregados enquanto LD*i* está a 1, com o interruptor ligado o *display* acompanha os interruptores de dados e, ao desligá-lo, fica com o último valor. O *clear* ficou na tecla central (BTNC), por ser uma ação momentânea.

Tabela 9: Ligação dos sinais do circuito de teste à Basys3.

Sinal	Pino(s)	Elemento da Basys3
CLKBasys	W5	Oscilador de 100 MHz
Data[3:0]	W17 W16 V16 V17	Interruptores SW3 a SW0
LD3 / LD2 / LD1 / LD0	R2 / T1 / U1 / W2	Interruptores SW15 / SW14 / SW13 / SW12
LDDP	R3	Interruptor SW11
EN	T2	Interruptor SW10
Clear	U18	Tecla BTNC
Seg[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a Low)
DP	V7	Ponto decimal (ativo a Low)
An[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a Low)

Além da localização dos pinos, o ficheiro de restrições define o relógio de 100 MHz (`create_clock`) e o relógio do varrimento gerado pelo divisor (`create_generated_clock`, com divisão por 50 000), para que a ferramenta saiba que os *flip-flops* do contador usam um relógio de 2 kHz. Numa primeira versão, o relatório de metodologia do Vivado (Report Methodology) apresentou um aviso crítico TIMING-8 (*No common period between related clocks*): o único caminho entre os dois relógios é a realimentação do próprio `ClockOut` do divisor, e o período comum dos dois relógios (500 µs, ou seja, 50 000 ciclos) é demasiado longo para a ferramenta o analisar. Esse caminho foi declarado como falso (`set_false_path`), tal como as entradas e saídas, que estão ligadas a interruptores, teclas e *displays* e não têm requisitos temporais. Com estas restrições, o relatório de metodologia deixou de ter avisos

críticos (resta apenas o aviso TIMING-9, que assinala precisamente o caminho entre os dois relógios declarado como falso). A implementação cumpre todas as restrições temporais, com uma folga mínima (WNS) de 5,020 ns no relógio de 100 MHz.

Listagem 23: Ficheiro de restrições do circuito de teste (`controlodisplays/testedisplays.xdc`)

```
# Ensaio do controlador dos displays na Basys3 (módulo de topo TesteDisplays, Fig. 32)

# Relógio interno da Basys3: 100 MHz
set_property PACKAGE_PIN W5 [get_ports CLKBasys]
    set_property IOSTANDARD LVCMOS33 [get_ports CLKBasys]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLKBasys]

# Relógio do varrimento, gerado pelo divisor (100 MHz / 50000 = 2 kHz)
create_generated_clock -name clk_varrimento -source [get_ports CLKBasys] -divide_by 50000 [get_pins controlo/divisor/ClockOut_reg/Q]
# O único caminho entre os dois relógios é a realimentação do próprio divisor
# (ClockOut_reg/Q -> inversor -> ClockOut_reg/D), sem período comum que o Vivado consiga analisar
set_false_path -from [get_clocks clk_varrimento] -to [get_clocks sys_clk_pin]

# Interruptores, teclas e displays são lentos e assíncronos em relação ao relógio:
# não há requisitos de tempo nas entradas e saídas
set_false_path -from [get_ports {Data[*] LD3 LD2 LD1 LD0 LDDP EN Clear}]
set_false_path -to [get_ports {Seg[*] DP An[*]}]

# Valor a carregar nos registos (interruptores SW3 a SW0)
set_property PACKAGE_PIN W17 [get_ports {Data[3]}]
set_property PACKAGE_PIN W16 [get_ports {Data[2]}]
set_property PACKAGE_PIN V16 [get_ports {Data[1]}]
set_property PACKAGE_PIN V17 [get_ports {Data[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Data[*]}]

# Carga dos registos dos símbolos (interruptores SW15 a SW12, alinhados com os displays:
# SW15 carrega o display da esquerda e SW12 o da direita)
set_property PACKAGE_PIN R2 [get_ports LD3]
    set_property IOSTANDARD LVCMOS33 [get_ports LD3]
set_property PACKAGE_PIN T1 [get_ports LD2]
    set_property IOSTANDARD LVCMOS33 [get_ports LD2]
set_property PACKAGE_PIN U1 [get_ports LD1]
    set_property IOSTANDARD LVCMOS33 [get_ports LD1]
set_property PACKAGE_PIN W2 [get_ports LD0]
    set_property IOSTANDARD LVCMOS33 [get_ports LD0]

# Carga do registo dos pontos decimais (interruptor SW11)
set_property PACKAGE_PIN R3 [get_ports LDDP]
    set_property IOSTANDARD LVCMOS33 [get_ports LDDP]

# Enable dos 4 displays (interruptor SW10)
set_property PACKAGE_PIN T2 [get_ports EN]
    set_property IOSTANDARD LVCMOS33 [get_ports EN]

# Clear de todos os registos (tecla BTNC)
set_property PACKAGE_PIN U18 [get_ports Clear]
    set_property IOSTANDARD LVCMOS33 [get_ports Clear]

# Segmentos a..g (Seg[6] = a ... Seg[0] = g), ativos a Low
set_property PACKAGE_PIN W7 [get_ports {Seg[6]}]
set_property PACKAGE_PIN W6 [get_ports {Seg[5]}]
set_property PACKAGE_PIN U8 [get_ports {Seg[4]}]
set_property PACKAGE_PIN V8 [get_ports {Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg[2]}]
set_property PACKAGE_PIN V5 [get_ports {Seg[1]}]
set_property PACKAGE_PIN U7 [get_ports {Seg[0]}]
```

```
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg[*]}]

# Ponto decimal, ativo a Low
set_property PACKAGE_PIN V7 [get_ports DP]
    set_property IOSTANDARD LVCMOS33 [get_ports DP]

# Ânodos comuns dos 4 displays, ativos a Low (An[0] é o display da direita)
set_property PACKAGE_PIN U2 [get_ports {An[0]}]
set_property PACKAGE_PIN U4 [get_ports {An[1]}]
set_property PACKAGE_PIN V4 [get_ports {An[2]}]
set_property PACKAGE_PIN W4 [get_ports {An[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {An[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* do circuito de teste (Listagem 24) reproduz o que se faz na placa: escolhe um valor nos interruptores de dados e liga e desliga o interruptor de carga do *display* pretendido. Ao contrário do anterior, vê o circuito apenas pelos pinos (caixa preta): em cada ciclo descobre qual é o *display* ativo a partir dos ânodos, confirma que há exatamente um, que os segmentos e o ponto são os que foram carregados nesse *display* e que a ordem do varrimento é 0, 1, 2, 3, 0, … Com `EN = 0` confirma que os quatro ânodos estão a 1. Na Figura 21 escreve-se "rdLE", acrescentam-se pontos nos *displays* 2 e 0 ("rd.LE."), substituem-se os dois últimos símbolos ("rd.C-."), desliga-se e volta a ligar-se `EN` (os valores mantêm-se) e, por fim, prime-se `Clear` (tudo a 0 e pontos apagados). Na vista completa os valores de `An` e `Seg` não cabem no ecrã; a Figura 22 amplia o intervalo em que os *displays* mostram "rd.LE.": `An` ativa um *display* de cada vez, `Seg` apresenta o símbolo carregado nesse *display* e `DP` vai a 0 nos *displays* 2 e 0. Resultado: `Simulacao OK: TesteDisplays sem erros (391 verificacoes)`.

Listagem 24: Testbench do circuito de teste (`controlodisplays/testedisplays_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do circuito de teste na placa (TesteDisplays, Fig. 32).
// Reproduz o que se faz na Basys3: escolher um valor nos interruptores Data,
// ligar e desligar o interruptor LDx do display pretendido, etc.
// Vê o circuito só pelos pinos (caixa preta): descobre o display ativo pelos
// ânodos e confirma que os segmentos e o ponto são os que foram carregados nele.
// METADE = 5: cada display fica ativo 100ns (na placa são 0,5 ms).
module testedisplays_teste1();

//   Declaração das entradas
     reg [3:0] Data = 0;
     reg LD3 = 0, LD2 = 0, LD1 = 0, LD0 = 0, LDDP = 0;
     reg EN = 0, Clear = 0;
     reg CLKBasys = 0;

//   Declaração das saídas
     wire [6:0] Seg;
     wire DP;
     wire [3:0] An;

//   Instanciação da UUT e ligação das entradas e saídas
     TesteDisplays #(.METADE(5)) UUT (
         .Data(Data),
         .LD3(LD3), .LD2(LD2), .LD1(LD1), .LD0(LD0), .LDDP(LDDP),
         .EN(EN),
         .Clear(Clear),
         .CLKBasys(CLKBasys),
```

```
29          .Seg(Seg),
30          .DP(DP),
31          .An(An)
32      );
33
34  //  Relógio de 100 MHz da Basys3
35      always #5 CLKBasys = ~CLKBasys;
36
37  //  Segmentos acesos (ativo a High) de cada símbolo
38      reg [6:0] acesos [0:15];
39      initial begin           //  abcdefg
40          acesos[0]  = 7'b1111110;  acesos[1]  = 7'b0110000;
41          acesos[2]  = 7'b1101101;  acesos[3]  = 7'b1111001;
42          acesos[4]  = 7'b0110011;  acesos[5]  = 7'b1011011;
43          acesos[6]  = 7'b1011111;  acesos[7]  = 7'b1110000;
44          acesos[8]  = 7'b1111111;  acesos[9]  = 7'b1111011;
45          acesos[10] = 7'b0000101;  acesos[11] = 7'b0111101;
46          acesos[12] = 7'b0001110;  acesos[13] = 7'b1001111;
47          acesos[14] = 7'b1001110;  acesos[15] = 7'b0000001;
48      end
49
50  //  O que deve estar guardado em cada display (atualizado como os registos)
51      reg [3:0] mostra [0:3];
52      reg [3:0] pontos = 0;
53      integer i;
54      initial for (i = 0; i < 4; i = i + 1) mostra[i] = 0;
55      always @(posedge CLKBasys or posedge Clear)
56          if (Clear) begin
57              for (i = 0; i < 4; i = i + 1) mostra[i] <= 0;
58              pontos <= 0;
59          end
60          else begin
61              if (LD0)  mostra[0] <= Data;
62              if (LD1)  mostra[1] <= Data;
63              if (LD2)  mostra[2] <= Data;
64              if (LD3)  mostra[3] <= Data;
65              if (LDDP) pontos    <= Data;
66          end
67
68  //  Verificação pelos pinos, a meio de cada ciclo de relógio
69      integer erros = 0, verificacoes = 0, k;
70      integer ativo, anterior = -1, visitas = 0;
71      always @(negedge CLKBasys) if ($time > 0) begin   // em t=0 a inicialização do relógio conta
    como flanco
72          verificacoes = verificacoes + 1;
73          if (!EN) begin
74              if (An !== 4'b1111) begin
75                  $display("ERRO t=%0t: EN=0 mas An=%b", $time, An);
76                  erros = erros + 1;
77              end
78          end
79          else begin
80              ativo = -1;
81              for (k = 0; k < 4; k = k + 1)
82                  if (An === ~(4'b0001 << k)) ativo = k;
83              if (ativo < 0) begin
84                  $display("ERRO t=%0t: An=%b (tem de haver exatamente um display ativo)", $time,
    An);
85                  erros = erros + 1;
86              end
87              else begin
88                  if (Seg !== ~acesos[mostra[ativo]] || DP !== ~pontos[ativo]) begin
```

```
89                      $display("ERRO t=%0t: display %0d mostra Seg=%b DP=%b (esperado Seg=%b DP=
    %b)",
90                              $time, ativo, Seg, DP, ~acesos[mostra[ativo]], ~pontos[ativo]);
91                      erros = erros + 1;
92                  end
93                  // o display ativo tem de seguir a ordem 0, 1, 2, 3, 0, ...
94                  if (anterior >= 0 && ativo != anterior && ativo != (anterior + 1) % 4) begin
95                      $display("ERRO t=%0t: o varrimento saltou do display %0d para o %0d", $time,
    anterior, ativo);
96                      erros = erros + 1;
97                  end
98                  if (ativo != anterior) visitas = visitas + 1;
99                  anterior = ativo;
100             end
101         end
102     end
103
104 //  Carregar um valor num display: escolher Data e ligar/desligar o interruptor LDx
105     task escreve(input integer display, input [3:0] valor);
106         begin
107             Data = valor; #20;
108             case (display)
109                 0: LD0 = 1;  1: LD1 = 1;  2: LD2 = 1;  3: LD3 = 1;  default: LDDP = 1;
110             endcase
111             #30;
112             {LD3, LD2, LD1, LD0, LDDP} = 5'b00000;
113             #20;
114         end
115     endtask
116
117 //  Definição do valor das entradas ao longo do tempo
118     initial begin
119         #202;                       // configurações iniciais da FPGA
120                                     // (as entradas mudam 2ns depois de um flanco descendente)
121         EN = 1;          #400;      // displays ligados: "0000"
122         escreve(3, 4'd10);          // r no display da esquerda
123         escreve(2, 4'd11);          // d
124         escreve(1, 4'd12);          // L
125         escreve(0, 4'd13);          // E
126                          #400;      // "rdLE"
127         escreve(4, 4'b0101);        // pontos nos displays 2 e 0: "rd.LE."
128                          #400;
129         escreve(1, 4'd14);          // C
130         escreve(0, 4'd15);          // -
131                          #400;      // "rd.C-."
132         Data = 4'd8;     #400;      // mudar Data sem carregar não altera nada
133         EN = 0;          #400;      // displays desligados
134         EN = 1;          #400;      // voltam a mostrar o mesmo
135         #3 Clear = 1;    #20 Clear = 0;
136                          #400;      // tudo a 0 e pontos apagados: "0000"
137
138         if (visitas < 30) begin
139             $display("ERRO: o varrimento so passou por %0d displays", visitas);
140             erros = erros + 1;
141         end
142         if (erros == 0) $display("Simulacao OK: TesteDisplays sem erros (%0d verificacoes)",
    verificacoes);
143         else            $display("Simulacao FALHOU: TesteDisplays com %0d erros", erros);
144         $finish;
145     end
146 endmodule
```

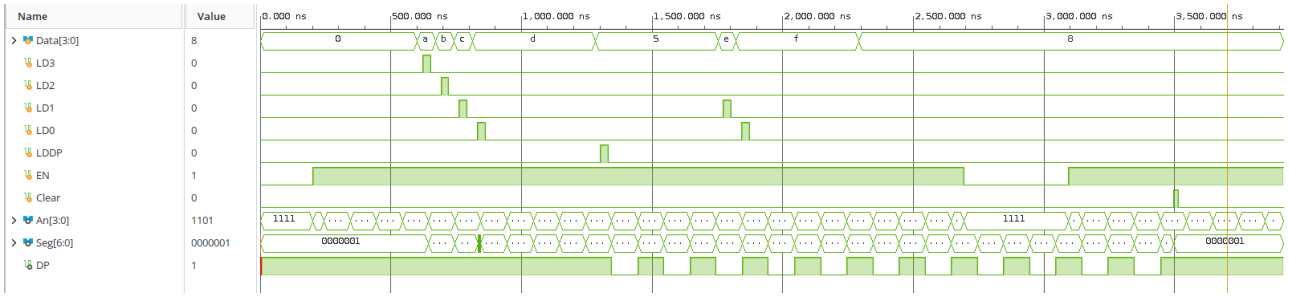


Figura 21: Simulação do circuito de teste `TesteDisplays` no Vivado (METADE = 5).

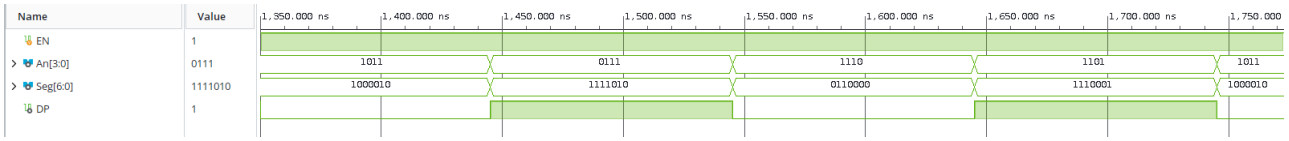


Figura 22: Pormenor da simulação do circuito de teste: "rd.LE." nos quatro displays.

A síntese, a implementação e a geração do *bitstream* (`controlodisplays.bit`) decorreram sem erros. O circuito ocupa 20 LUT e 55 *flip-flops* (20 dos cinco registos, 32 do contador do divisor, o próprio `ClockOut` e os 2 do contador de módulo 4). O ensaio na placa seguiu este procedimento:

1. ligar SW10 (EN): os quatro *displays* mostram 0;
2. para cada *display*, colocar o valor em SW3 a SW0 e ligar e desligar o respetivo interruptor (SW15 para o da esquerda até SW12 para o da direita): com 1, 2, 3 e 4 os *displays* mostram "1234";
3. com SW3 a SW0 = 1011 ligar e desligar SW11: acendem os pontos decimais dos *displays* 3, 1 e 0 ("1.23.4.", Figura 23 (a));
4. da mesma forma, carregar os símbolos 10 ("r"), 15 ("-"), 8 e 11 ("d") e os pontos com 0010: os *displays* mostram "r-8.d", com o ponto decimal só no *display* 1 (Figura 23 (b));
5. premir BTNC (Clear): os cinco registos voltam a 0 e os quatro *displays* mostram 0, sem pontos (Figura 24).

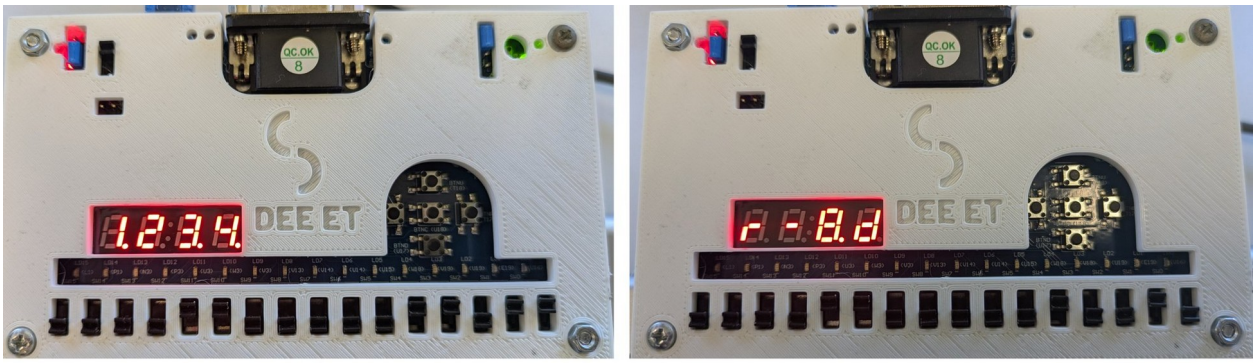


Figura 23: Ensaio do controlador na placa Basys3: (a) "1.23.4."; (b) "r-8.d", com símbolos originais.

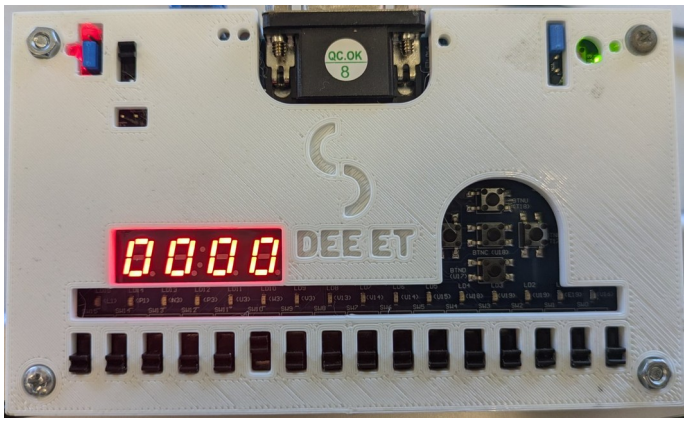


Figura 24: Ensaio do controlador na placa Basys3 depois do Clear (BTNC): "0000", sem pontos.

9 Análise crítica do código de fontes externas

Nesta secção analisa-se o código Verilog que não foi escrito de raiz: o código gerado pelo Digital, os exemplos Verilog disponibilizados no Moodle da unidade curricular (que, como indica o enunciado, têm erros introduzidos intencionalmente), o modelo da primitiva FDCE e o código produzido com apoio de IA generativa.

9.1 Código gerado pelo Digital

Os ficheiros `multiplexer4x1.v` e `descodificador2x4.v` foram gerados pelo Digital e usados sem alterações (o próprio Digital avisa no cabeçalho que o ficheiro não deve ser modificado, porque as alterações se perdem quando é exportado de novo). O código é correto e legível: uma atribuição contínua por saída, com a mesma expressão lógica do esquema. Os nomes das portas são os nomes dados no esquema, o que mostra a importância de escolher nomes simples e com as maiúsculas certas, já que o Verilog distingue maiúsculas de minúsculas. As simulações no Digital e os *testbenches* no Vivado confirmaram que a exportação não alterou a função dos circuitos.

9.2 Transcodificador do Moodle

O módulo `Transcod7Seg` do Moodle tem a estrutura certa (um `case` sobre `Numero` dentro de um bloco `always`), mas não compila nem cumpre o pretendido sem as correções da Tabela 10.

Tabela 10: Erros do transcodificador do Moodle e correções introduzidas.

Código do Moodle	Problema	Correção
4'b1111 : seg <= ...	O Verilog distingue maiúsculas de minúsculas: seg não está declarado (o registo chama-se Seg) e o código não compila.	Atribuição feita sempre à mesma variável (Segmentos).
default : segmentos <= ...	Também segmentos (com minúscula) não está declarado e o código não compila.	Atribuição feita a Segmentos, declarada diretamente como output reg [6:0] (dispensa o registo auxiliar Seg e o assign).
reg [7:0] Seg;	8 *bits* para 7 segmentos: o *bit* 7 nunca é usado e a ligação a Segmentos[6:0] trunca o valor (aviso de larguras diferentes).	Removido o registo auxiliar; a saída tem 7 *bits*.
4'b1001 : Seg <= 7'b0001100;	O algarismo 9 fica sem o segmento *d*, ao contrário da Fig. 30.	7'b0000100 (segmentos a b c d f g).
Falta o caso 4'b1110	O valor 14 cai no default e o *display* fica apagado.	Os 16 valores têm um caso próprio.
4'b1011 : Seg <= 7'b1001111;	O símbolo 11 é igual ao algarismo 1 (ambíguo no *display*).	Novos símbolos 10 a 15, todos diferentes ("rdLEC-").
always @(Numero) com <=	Funciona, mas num circuito combinatório é preferível always @(*) (dispensa a manutenção da lista de sensibilidade) com atribuições bloqueantes.	always @(*) com =.

Os símbolos 10 a 15 do Moodle (por exemplo "≡" e "°") foram todos substituídos pelos símbolos originais descritos na secção 5, como pede o enunciado. A verificação de que os 16 símbolos são diferentes foi incluída no *testbench* precisamente para detetar ambiguidades como a do símbolo 11 original.

9.3 Divisores de clock do Moodle

O Moodle tem dois divisores de *clock*. O primeiro (`clk_div`) usa atribuições bloqueantes (`a=a+1`) num bloco sequencial, o que funciona neste caso, mas é uma má prática (pode criar diferenças entre a simulação e o circuito sintetizado quando há vários blocos a ler as mesmas variáveis); o comentário indica 1 kHz para um relógio de 25 MHz, o que na Basys3 (100 MHz) daria 4 kHz. O segundo (`Clock_Divider`) tem os seguintes problemas:

- `HALFCYCLE` está definido com o valor de teste 10 (o valor "real" está comentado), pelo que, sem alteração, a saída teria cerca de 4,5 MHz;
- o contador vai de 0 a `HALFCYCLE` antes de inverter a saída, ou seja, cada meio período dura `HALFCYCLE + 1` ciclos e não `HALFCYCLE` (erro de uma unidade);
- a indentação sugere que o último `else` pertence a `if (Enable)`, mas em Verilog um `else` pertence sempre ao `if` mais próximo; neste caso o comportamento acaba por ser o pretendido, mas o código é enganador;
- o contador é de 32 *bits*, mas as constantes são de 25 *bits* (`25'd0`), e o bloco `initial` repete o que o *reset* já faz.

O módulo `DivisorClock` mantém a ideia (contador e inversão da saída), retira o *enable* e o *reset*, que não são necessários neste projeto, usa atribuições não bloqueantes e compara com `METADE - 1`, para cada meio período durar exatamente `METADE` ciclos. A mutação "conta METADE+1" do teste de mutações (secção 9.6) confirma que o *testbench* deteta o erro de uma unidade do exemplo original.

9.4 Contadores do Moodle

O "contador simples" do Moodle conta módulo 4 com um registo de 2 *bits*, mas tem uma saída de 4 *bits* (os 2 *bits* mais significativos ficam sempre a 0), usa uma atribuição bloqueante no bloco sequencial e um segundo bloco `always` apenas para copiar o estado para a saída. O "contador em anel" nem sequer compila: a lista de portas declara `Qd`, mas a porta declarada no corpo do módulo é `Q`. O módulo `Contador4` ficou reduzido ao essencial: um registo de 2 *bits* com atribuição não bloqueante e as saídas `Q1` e `Q0` ligadas diretamente aos seus *bits*. Note-se que o contador em anel não é necessário neste projeto, porque a sua função (ativar uma de quatro saídas) é feita pelo descodificador 2x4.

9.5 Primitiva FDCE

A instanciação da FDCE segue o modelo do próprio Vivado (Language Templates). O único cuidado foi confirmar a polaridade dos sinais: na FDCE o `CLR` é ativo a *High* e assíncrono e o `CE` é ativo a *High*, que é o pretendido para o registo. Como o Icarus Verilog (usado no teste de mutações) não tem a biblioteca UNISIM, foi escrito um modelo simples da FDCE apenas para essa ferramenta (`scripts/fdce_modelo.v`); todas as simulações apresentadas neste relatório foram feitas no Vivado, com a primitiva verdadeira.

9.6 IA generativa

Foi usada IA generativa (Claude, da Anthropic) como apoio na escrita de parte do código Verilog, dos *testbenches* e dos *scripts* de automatização (criação dos projetos Vivado, simulação, geração dos *bitstreams* e das vistas das formas de onda) e na redação deste relatório. Todo o código foi verificado por simulação no Vivado e, adicionalmente, por um **teste de mutações** (`scripts/teste_mutacoes.py`): introduziram-se, um de cada vez, 28 erros típicos numa cópia dos módulos (por exemplo, *clear* síncrono, flanco descendente, ânodos não negados, registos trocados, *enables* trocados) e confirmou-se que o *testbench* respetivo falha em todos. Este processo revelou problemas reais, que foram corrigidos:

- a primeira versão do *testbench* do controlador carregava sempre os quatro registos em simultâneo, pelo que não detetava um erro de ligação em que `Load3` carregasse também o registo 2; passou a carregar cada registo sozinho, com valores diferentes nas outras entradas;
- as entradas mudavam exatamente nos flancos descendentes, no mesmo instante em que o *testbench* verificava as saídas, o que dava resultados dependentes da ordem de execução do simulador; passaram a mudar 2 ns depois dos flancos descendentes (3 ns antes do flanco ascendente seguinte);
- no simulador do Vivado, a inicialização do relógio do *testbench* em t = 0 é vista como um flanco descendente, quando as saídas ainda estão indefinidas (X); o Icarus Verilog não mostrava este

problema, o que confirma a importância de simular na ferramenta final. As verificações passaram a começar depois de t = 0;

- uma leitura literal da Fig. 31 (ânodos ligados diretamente ao descodificador) daria ânodos com a polaridade errada para a Basys3; a decisão de negar os ânodos foi confirmada no manual da placa e é verificada pelo *testbench* (mutação "ânodos não negados").

A conclusão é que o código produzido com apoio de IA, tal como o do Moodle, não pode ser aceite sem verificação: os *testbenches* com verificação automática e o teste de mutações foram essenciais para encontrar estes problemas.

10 Conclusão

Neste trabalho percorreu-se o fluxo completo de projeto de um sistema digital em FPGA: desenho e simulação no Digital, exportação para Verilog, criação de projetos no Vivado, escrita de *testbenches*, simulação comportamental, síntese, implementação, geração do *bitstream* e programação da Basys3. Foram desenvolvidos e validados um *multiplexer* 4x1, um descodificador 2x4 com *enables*, um *multiplexer* 16x4 construído por hierarquia de módulos, um transcodificador para *display* de 7 segmentos com os símbolos originais "rdLEC-", um registo de 4 *bits* baseado na primitiva FDCE, um divisor de *clock* parametrizável e um contador de módulo 4, que foram integrados no controlador dos quatro *displays* da placa e no respetivo circuito de teste.

O trabalho permitiu consolidar a distinção entre descrição comportamental e descrição estrutural em Verilog, a reutilização de módulos por hierarquia e a importância de *testbenches* que verificam automaticamente os resultados (todos os 9 *testbenches* terminam com `Simulacao OK`). Ficou também evidente o cuidado necessário com a polaridade dos sinais da placa (segmentos, pontos decimais e ânodos são ativos a *Low*) e com as restrições temporais quando se usa um relógio gerado pela lógica.

Todos os circuitos foram também ensaiados na placa Basys3, com os resultados esperados em todos os casos, como mostram as fotografias incluídas em cada secção.

Anexo A: Organização dos ficheiros

Os ficheiros do trabalho estão organizados com uma pasta por módulo, que contém o código, o *testbench* (`<nome>_teste1.v`), a vista das formas de onda (`.wcfg`) e, quando o módulo é ensaiado na placa, o ficheiro de restrições (`.xdc`):

Listagem 25: Estrutura das pastas do trabalho

```
sd2lab1/
├── digital/            # esquemas do Digital (.dig e .svg) e Verilog exportado
├── multiplexer4x1/     # secção 2
│   ├── multiplexer4x1.v
│   ├── multiplexer4x1.xdc
│   ├── multiplexer4x1_teste1.v
│   └── multiplexer4x1_teste1.wcfg
├── descodificador2x4/  # secção 3
│   ├── descodificador2x4.v
│   ├── descodificador2x4.xdc
│   ├── descodificador2x4_teste1.v
│   └── descodificador2x4_teste1.wcfg
├── multiplexer16x4/    # secção 4
│   ├── multiplexer16x4.v
│   ├── multiplexer16x4.xdc
│   ├── multiplexer16x4_teste1.v
│   └── multiplexer16x4_teste1.wcfg
├── transcod7seg/       # secção 5 (transcodificador e circuito displayBCD)
│   ├── displayBCD.v
│   ├── displayBCD.xdc
│   ├── transcod7seg.v
│   ├── transcod7seg_teste1.v
│   └── transcod7seg_teste1.wcfg
├── registo4/           # secção 6
│   ├── registo4.v
│   ├── registo4_teste1.v
│   └── registo4_teste1.wcfg
├── divisorclock/       # secção 7.1
│   ├── divisorclock.v
│   ├── divisorclock_teste1.v
│   └── divisorclock_teste1.wcfg
├── contador4/          # secção 7.2
│   ├── contador4.v
│   ├── contador4_teste1.v
│   └── contador4_teste1.wcfg
├── controlodisplays/   # secção 8 (Fig. 31 e Fig. 32)
│   ├── controlodisplays.v
│   ├── controlodisplays_teste1.v
│   ├── controlodisplays_teste1.wcfg
│   ├── testedisplays.v
│   ├── testedisplays.xdc
│   ├── testedisplays_teste1.v
│   └── testedisplays_teste1.wcfg
├── simulacao/          # formas de onda (VCD) de todas as simulações
├── bitstreams/         # ficheiros .bit e relatórios do Vivado
├── vivado/             # projetos Vivado (.xpr)
├── scripts/            # automatização (Tcl e Python)
├── relatorio/          # este relatório e as figuras
└── README.md
```

Os projetos Vivado (`vivado/<projeto>/<projeto>.xpr`) referem os ficheiros das pastas dos módulos por caminhos relativos e podem ser abertos com File > Project > Open. Para os recriar de raiz, executar

todas as simulações e gerar todos os *bitstreams*, basta correr `vivado -mode batch -source scripts/build.tcl` na pasta do trabalho. A Tabela 11 indica o conteúdo de cada projeto.

Tabela 11: Projetos Vivado do trabalho.

Projeto	Topo	*Testbenches*	LUT / FF
multiplexer4x1	multiplexer4x1	multiplexer4x1_teste1	1 / 0
descodificador2x4	descodificador2x4	descodificador2x4_teste1	2 / 0
multiplexer16x4	multiplexer16x4	multiplexer16x4_teste1	4 / 0
displayBCD	displayBCD	transcod7seg_teste1	8 / 0
controlodisplays	TesteDisplays	registo4_teste1, divisorclock_teste1, contador4_teste1, controlodisplays_teste1, testedisplays_teste1	20 / 55